

SỔ TAY KIẾN TRÚC VÀ VẬN HÀNH

APACHE AIRFLOW PYSPARK RETAIL

Từ yêu cầu bài đến hệ thống chạy thực tế

Tài liệu tiếng Việt dành cho người mới, nhóm phát triển và người bảo vệ bài. Bao gồm cơ sở lý thuyết, kiến trúc triển khai, từng task và thuật toán, hướng dẫn từ đầu, xử lý sự cố và bằng chứng nghiệm thu.

Phạm vi	Thông tin
Project	Airflow-Apache-Retail-E-Commerce-Dataset
Ngày chốt bằng chứng	29/09/2026
Runtime	Airflow 3.3.1 • PySpark 4.2.0 • PostgreSQL 16
Run kiểm chứng	official_smoke_20260929 • 6/6 task success
Kiểm thử	87 tests passed • 8 Parquet outputs read-back
Yêu cầu	airflow_subject.docx + rubric cập nhật 25/25/25/15/10

Bản chỉnh sửa chính: DOCX | Bản xem/in: PDF đi kèm

MỤC LỤC

Chọn tiêu đề để đi đến phần cần đọc. Trong Word có thể mở Navigation Pane bằng Ctrl+F.

1. Mục tiêu, phạm vi và cách sử dụng tài liệu	7
1.1 Bài toán mà project giải quyết	7
1.2 Nguồn yêu cầu và thứ tự ưu tiên	7
1.3 Phiên bản và trạng thái được mô tả.....	7
1.4 Quy ước đọc	8
2. Bản đồ kiến trúc toàn hệ thống.....	9
2.1 Hệ thống nhìn từ máy người dùng	9
2.2 Luồng điều khiển và luồng dữ liệu	9
2.3 Các ranh giới cần hiểu đúng	10
2.4 Một lượt chạy từ đầu đến cuối	10
3. Orchestration Plane và Compute Plane	11
3.1 Airflow làm gì, Spark làm gì?	11
3.2 Vì sao không xử lý toàn bộ CSV bằng pandas trong PythonOperator?	11
3.3 Vì sao chọn SparkSubmitOperator?	11
3.4 So sánh các mô hình tích hợp khi mở rộng	11
4. Các khái niệm Airflow được dùng trong bài.....	13
4.1 DAG, DagRun, Task và TaskInstance.....	13
4.2 Operator và Sensor.....	13
4.3 XCom, Params và Variables	13
4.4 Connection và Hook	13
4.5 Jinja và runtime context	13
4.6 Scheduler, Executor, DAG processor và API server	14
4.7 Retry, trigger rule và state.....	14
5. Docker và các thành phần triển khai.....	15
5.1 Image, container, build và Compose	15
5.2 Vai trò từng service	15
5.3 Volume, network và persistence	15
5.4 Dependency và cấu hình Python	16
5.5 Tài nguyên và cơ chế giới hạn	16
6. Bản đồ repository và trách nhiệm của từng phần.....	17
6.1 Cấu trúc module nghiệp vụ	17
6.2 Thành phần nghiên cứu, kiểm thử và tài liệu.....	17
6.3 Thành phần hỗ trợ cộng tác	17

6.4 Danh mục file và thư mục cần biết.....	17
7. Ngày nghiệp vụ, lịch chạy và backfill	19
7.1 Bốn khái niệm ngày khác nhau	19
7.2 Timetable và giới hạn lịch sử.....	19
7.3 Manual, scheduled và backfill	19
7.4 Vì sao snapshot tích lũy?	20
7.5 Quy trình backfill thận trọng	20
8. Thiết kế chi tiết sáu task.....	21
8.1 Bảng contract của từng task.....	21
8.2 start_pipeline và validate_raw_data.....	21
8.3 Ba SparkSubmitOperator.....	21
8.4 notify_completion	22
8.5 Retry và các mốc thời gian	22
8.6 SLA, DeadlineAlert và freshness	22
9. Dữ liệu nguồn và landing bất biến.....	23
9.1 Dataset và schema.....	23
9.2 Bootstrap làm những gì?	23
9.3 Bố cục landing.....	23
9.4 Các lớp bảo vệ nguồn	24
9.5 Dòng undated và giới hạn thời gian	24
10. PySpark ETL và Partitioned Parquet	25
10.1 Parse tường minh, không schema inference	25
10.2 Dedup và hai chính sách giữ dữ liệu.....	25
10.3 Ghi dữ liệu và ý nghĩa partition	25
10.4 Spark execution và hiệu năng.....	26
10.5 Các đối soát ETL đã quan sát	26
11. RFM, phân khúc và inactivity proxy.....	27
11.1 Định nghĩa toán học	27
11.2 Ví dụ tính bằng tay.....	27
11.3 Tính điểm R/F/M bằng Window	27
11.4 KMeans và lựa chọn k.....	27
11.5 Quy tắc nhãn khách hàng	28
11.6 Churn risk là proxy có điều kiện	28
12. Chất lượng dữ liệu và phát hiện bất thường.....	30
12.1 Vì sao cần hai cấp audit?	30
12.2 Ba nhóm thông tin trên mỗi dòng.....	30

12.3 Sáu quy tắc nghiệp vụ trên dòng.....	30
12.4 Ba quy tắc IQR theo sản phẩm	31
12.5 Từ khóa trong Description và bằng chứng tên sản phẩm	31
12.6 Trạng thái và bảo toàn dòng.....	31
12.7 Ngưỡng thống kê trên tổng hóa đơn	31
12.8 Ví dụ và kết quả thực tế	32
13. Manifest, versioning và tính nhất quán khi chạy lại.....	33
13.1 Ba tầng metadata	33
13.2 Định danh và fingerprint	33
13.3 Idempotency đúng nghĩa trong project.....	33
13.4 Atomic publication và giới hạn	33
13.5 Đọc kết quả đúng cách	34
14. Hướng dẫn cài đặt từ máy mới	35
14.1 Chuẩn bị và kiểm tra vị trí làm việc	35
14.2 Tạo .env đúng nơi và đúng mục đích	35
14.3 Build và khởi tạo dịch vụ	36
14.4 Lấy dữ liệu	36
14.5 Bootstrap, connections và pool.....	36
14.6 Kiểm tra môi trường trước lần chạy đầu.....	36
14.7 Chạy sau khi đã cài rồi	37
15. Chạy, theo dõi và sử dụng kết quả	38
15.1 Trigger qua giao diện	38
15.2 Trigger bằng PowerShell không lỗi dấu ngoặc JSON	38
15.3 Cách đọc log theo tầng.....	38
15.4 Bằng chứng lần chạy thật	39
15.5 Đọc Parquet bằng Spark	39
15.6 Chạy script legacy và khác biệt output.....	39
15.7 Dừng và chạy lại dịch vụ	40
16. Kiểm thử, đối soát và bằng chứng nghiệm thu	41
16.1 Các lớp kiểm thử	41
16.2 Bộ test và giới hạn fixture	41
16.3 Kết quả kiểm chứng đã có	43
16.4 Những kiểm thử chưa được làm	43
16.5 Bộ bằng chứng để nộp	44
17. Xử lý sự cố và phục hồi.....	45
17.1 Quy trình chẩn đoán theo thứ tự	45

17.2 Các lệnh thu thập thông tin không thay dữ liệu.....	45
17.3 Chọn retry, clear hay run mới	46
17.4 Sao lưu và dọn dữ liệu.....	46
18. Bảo mật, tái lập và giới hạn mở rộng	47
18.1 Các biên bảo mật thực tế	47
18.2 Reproducibility có những tầng nào?	47
18.3 Các điểm nghẽn khi tăng dữ liệu	47
18.4 SLO và observability tiếp theo.....	47
19. Tổng hợp kết quả và ma trận đáp ứng yêu cầu.....	49
19.1 Kết quả từ artifact thực tế.....	49
19.2 Ánh xạ yêu cầu chức năng của DOCX	49
19.3 Đáp ứng rubric mới mà không tự chấm điểm	50
19.4 Phần còn lại trước bài nộp cuối.....	50
20. Kịch bản demo và câu hỏi bảo vệ	51
20.1 Kịch bản khoảng 15 phút.....	51
20.2 Các câu hỏi nên trả lời được.....	51
20.3 Bài thực hành đọc code.....	51
21. Nguồn tham khảo và căn cứ thiết kế.....	52
21.1 Sách người dùng cung cấp.....	52
21.2 Tài liệu chính thức và nguồn local	52
Phụ lục A. Từ điển thuật ngữ và cấu hình nhanh	54
Phụ lục B. Từ điển dữ liệu đầu ra thực tế.....	56
audit / flagged_orders.....	56
audit / order_assessments	56
audit / row_assessments.....	57
etl / audit_input	58
etl / parse_errors.....	58
etl / transactions	58
rfm / cluster_profile	59
rfm / customers.....	59
Phụ lục C. Danh mục hàm và điểm vào của code.....	60
dags/ecommerce_etl_dag.py.....	60
dags/retail_support/tasks.py	60
scripts/retail_contracts.py	60
scripts/prepare_daily_landing.py.....	61
scripts/retail_pipeline.py.....	61

scripts/pyspark_clean.py.....	61
scripts/pyspark_rfm.py	61
scripts/pyspark_anomalies.py.....	62
scripts/select_kmeans_k.py	62
Phụ lục D. Cấu hình triển khai và DAG đầy đủ.....	63
D.1 Dockerfile	63
D.2 Docker Compose	63
D.3 DAG chính thức	64
D.4 Cấu hình lint	66
Phụ lục E. Quy trình cập nhật tài liệu	67

1. Mục tiêu, phạm vi và cách sử dụng tài liệu

1.1 Bài toán mà project giải quyết

Project xây dựng một pipeline bán lẻ theo lô: nhận giao dịch, kiểm tra đầu vào, làm sạch bằng PySpark, tính chỉ số khách hàng, rà soát giao dịch bất thường và công bố kết quả có thể truy vết. Airflow chịu trách nhiệm quyết định bước nào được chạy, khi nào được chạy, chạy lại ra sao và ghi nhận thành công/thất bại. Spark chịu trách nhiệm đọc, biến đổi, tổng hợp và ghi dữ liệu.

Một người mới có thể dùng tài liệu này theo hai tuyến. Tuyến thực hành: đọc chương 2 để hình dung hệ thống, chương 14 để cài đặt, chương 15 để chạy và chương 17 để xử lý lỗi. Tuyến nghiên cứu/bảo vệ: đọc lần lượt khái niệm Airflow, quyết định kiến trúc, hợp đồng dữ liệu, thuật toán và ma trận yêu cầu. Các phụ lục giúp tra cứu cấu hình và tìm đúng hàm trong code.

Đơn vị xử lý nhỏ nhất của CSV là một dòng hóa đơn, không phải một khách hàng hay một đơn hàng. Một hóa đơn có nhiều dòng sản phẩm; một khách hàng có nhiều hóa đơn. Phân biệt ba cấp này là điều kiện để tính đúng Frequency, tổng đơn và số lượng đối tượng bị gắn cờ.

1.2 Nguồn yêu cầu và thứ tự ưu tiên

Tài liệu dùng mục 1–5 của [airflow_subject.docx](#) làm yêu cầu chức năng, bảng rubric 25/25/25/15/10 do người dùng cập nhật làm tiêu chí đánh giá, và code cùng bằng chứng chạy làm căn cứ mô tả hiện trạng. [requirements.txt](#) là danh sách dependency chạy pipeline; [requirements-dev.txt](#) phục vụ kiểm tra code. Đây là hai nghĩa khác nhau của “requirements”, và cả hai đều được đối chiếu.

Mục rubric Kafka/Streaming trong bản DOCX cũ đã được thay thế bằng bảng cập nhật của người dùng. Vì vậy Kafka, Flume, watermark, sliding window, recommendation và Cassandra không phải thành phần đang có của project này. Tài liệu vẫn giải thích hướng mở rộng khi phù hợp, nhưng không vẽ những dịch vụ đó như thể đã được triển khai.

Nhóm đánh giá hiện hành	Điểm	Nội dung tài liệu và bằng chứng tương ứng
Conceptual & Theoretical Depth	25	DAG, Operator, XCom, backfill; hai lớp điều phối/tính toán; lý do tích hợp Spark
Airflow DAG Implementation	25	Sáu task thật; interval; Sensor; retry; deadline; phụ thuộc và trạng thái
PySpark ETL & Analytics Jobs	25	Schema, cleaning, DataFrame/Window RFM, Partitioned Parquet
Architecture, Setup & Reproducibility	15	Docker Compose, cấu trúc module, hướng dẫn chạy, lint, logs và bằng chứng
Data Quality & Anomaly Detection	10	Null/bad records, quarantine, audit dòng và ngưỡng tổng hóa đơn

1.3 Phiên bản và trạng thái được mô tả

Bản tài liệu chụp lại trạng thái đã kiểm chứng ngày 29/09/2026. Image project là [ecommerce-airflow:3.3.1](#); Airflow 3.3.1, PySpark 4.2.0, Spark provider 6.3.2, standard provider 1.17.0 và FAB provider 3.8.0. PostgreSQL dùng image major [postgres:16](#); Dockerfile cài Java 17. Các phiên bản ứng dụng cụ thể được ghi lại để tránh sao chép hướng dẫn Airflow 2 vào Airflow 3.

Lần chạy [official_smoke_20260929](#) đã thành công cả sáu task. Có 87 test đạt và có bước đọc lại tám output Parquet độc lập. Điều này chứng minh pipeline thực tế hoạt động trên dữ liệu mẫu đã chọn; không

tự chứng minh khả năng chạy hàng triệu dòng mỗi ngày, cluster nhiều máy, hoặc mọi tình huống mất điện/mất worker.

1.4 Quy ước đọc

“Đã triển khai” nghĩa là có code trong repository. “Đã kiểm chứng” đi kèm lệnh, kết quả hoặc artifact thực tế. “Mở rộng” là hướng phát triển chưa có trong Compose/DAG hiện tại. Sơ đồ trong tài liệu là sơ đồ kỹ thuật được dựng từ cấu hình; biểu đồ thời gian dựng từ trạng thái task đã ghi nhận, không phải ảnh chụp giao diện Airflow.

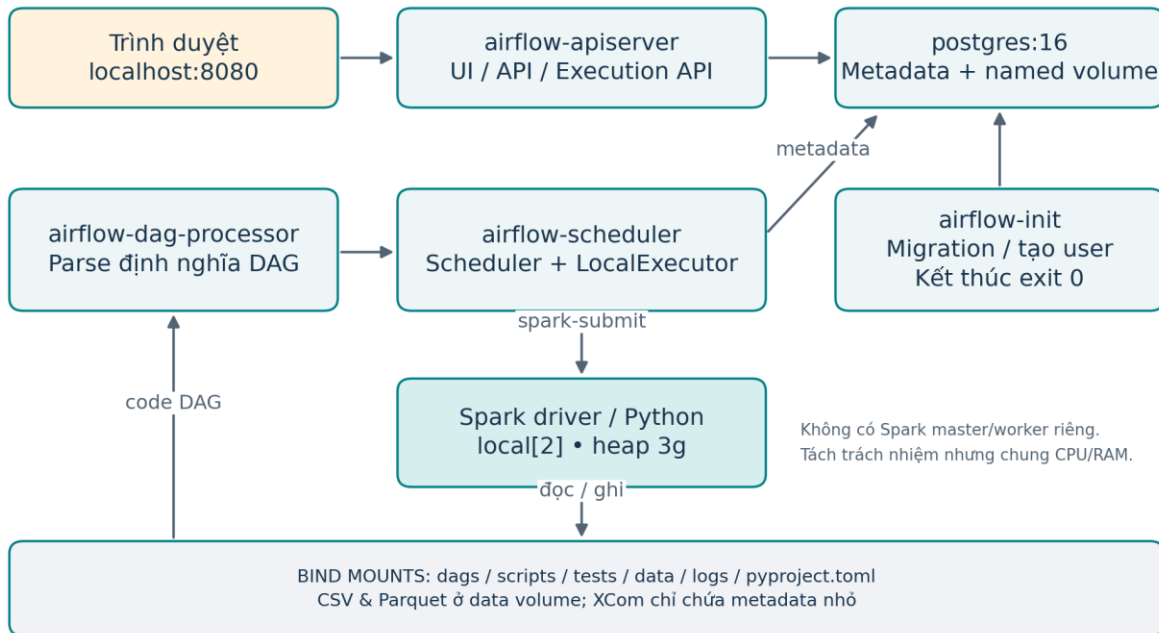
Các giá trị trong dấu `<...>` là chỗ người vận hành tự thay. Không có mật khẩu, Fernet key hoặc JWT secret thực tế của người dùng trong tài liệu. Toàn bộ lệnh chính dùng PowerShell tại thư mục chứa `docker-compose.yaml`, trừ nơi ghi rõ chạy trong container.

2. Bản đồ kiến trúc toàn hệ thống

2.1 Hệ thống nhìn từ máy người dùng

Máy Windows chạy Docker Desktop với Linux containers. Docker chạy PostgreSQL và các tiến trình Airflow riêng. Người dùng truy cập trình duyệt tại <http://localhost:8080>; cổng này được ánh xạ tới API server của Airflow. Source code và dữ liệu nằm trong repository trên Windows, được bind mount vào các container ở `/opt/airflow`.

WINDOWS HOST • DOCKER DESKTOP / LINUX CONTAINERS



Hình 1. Kiến trúc triển khai thực tế: Docker, các dịch vụ Airflow, PostgreSQL và Spark local.

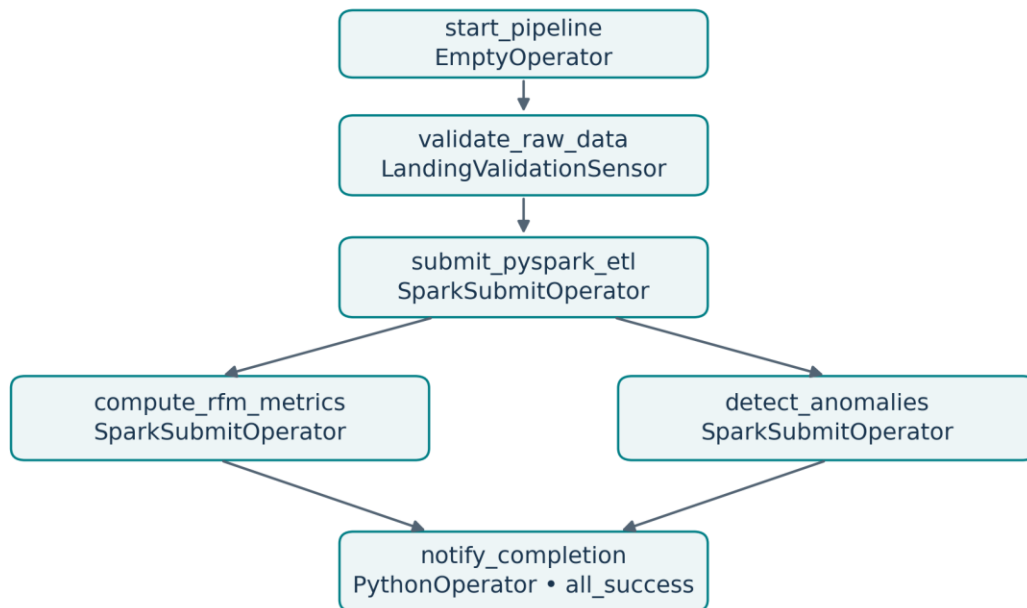
API server phục vụ giao diện/API, gồm endpoint phục vụ task execution. DAG processor đọc code DAG và cập nhật thông tin để scheduler lập lịch. Scheduler kiểm tra phụ thuộc, giới hạn concurrency và pool; LocalExecutor thực thi task trên cùng môi trường scheduler. Khi đến task Spark, SparkSubmitOperator gọi `spark-submit`, tạo tiến trình Spark driver/JVM và Python tương ứng trong container scheduler.

Không có container Spark master/worker riêng trong Compose. `local[2]` nghĩa là Spark local sử dụng hai luồng thực thi cho job, không phải hai máy worker hay hai container. Không có Redis, Celery worker, Flower, Livy, YARN ResourceManager, Kubernetes executor hoặc triggerer trong cấu hình project này.

2.2 Luồng điều khiển và luồng dữ liệu

Luồng điều khiển là các quyết định của Airflow: tạo DagRun, kiểm tra task upstream, chuyển trạng thái, cấp slot, submit Spark, nhận exit code, retry và callback. Luồng dữ liệu là CSV/Parquet/manifest trên volume. Hai luồng liên quan nhưng không đồng nhất: một dependency `A >> B` không tự chuyển DataFrame từ A sang B.

Thông tin nhỏ đi qua XCom từ validation tới các task Spark: đường dẫn context và metadata. Dữ liệu giao dịch đi qua các file có đường dẫn trong stage manifest. Một task chỉ được coi là thành công khi tiến trình hoàn tất không lỗi; kết quả chỉ được công bố cho consumer sau khi bước completion kiểm tra cả ba stage.



Hai nhánh độc lập; pool retail_spark=1 nên chạy Spark lần lượt.

Hình 2. DAG sáu task; fan-out sau ETL và fan-in trước công bố kết quả.

2.3 Các ranh giới cần hiểu đúng

Ranh giới	Thành phần hai bên	Ý nghĩa
Máy host / container	PowerShell, repository / Linux runtime	Đường dẫn Windows khác đường dẫn trong container; volume nối hai bên
Điều phối / tính toán	Airflow / Spark driver và tasks	Tách trách nhiệm ở code và process; vẫn chia sẻ tài nguyên máy
Metadata / business data	PostgreSQL, XCom / CSV, Parquet	Database Airflow lưu lịch và trạng thái, không phải kho giao dịch bán lẻ
Nguồn / kết quả	CSV gốc, landing / curated, analytics, audit	Không ghi đè nguồn; mỗi kết quả có provenance
Kết quả đang ghi / đã công bố	Generation chưa commit / published manifest	Consumer đọc đúng generation hoàn chỉnh
Nghiên cứu / pipeline hằng ngày	Notebook, chọn k / sáu task DAG	Notebook giúp chọn quy tắc; không được scheduler tự chạy

2.4 Một lượt chạy từ đầu đến cuối

Ví dụ người dùng trigger với D=2011-12-09. Validation chờ manifest `online-retail-v1`, kiểm tra đầy đủ lịch sử từ ngày đầu đến D, rồi viết context có cutoff C=2011-12-10. ETL xác minh checksum, parse dữ liệu và ghi ba output. RFM đọc output clean; audit đọc output ít lọc hơn của cùng ETL. Mỗi nhánh ghi stage manifest riêng. Completion kiểm tra hai nhánh cùng sử dụng đúng ETL generation, rồi xuất một published manifest.

Người phân tích dùng published manifest tìm output `customers`, `row_assessments`, `order_assessments` hoặc `flagged_orders`. Không chọn thư mục có tên mới nhất bằng mắt vì nó có thể thuộc lần thử chưa hoàn tất.

3. Orchestration Plane và Compute Plane

3.1 Airflow làm gì, Spark làm gì?

Công việc	Airflow	PySpark/Spark
Xác định lịch và dependency	DAG, timetable, scheduler	Không thay Airflow quyết định luồng nghiệp vụ nhiều job
Chờ nguồn và kiểm tra metadata	FileSensor, validation helper	Xác minh đầy đủ checksum trước đọc và schema khi xử lý
Tổng hợp dữ liệu lớn	Chỉ submit và theo dõi	DataFrame, aggregation, join, Window, ML
Phân chia công việc tính toán	LocalExecutor cấp task Airflow	Spark chia job thành stage/task và xử lý partition
Lỗi và chạy lại	Trạng thái, retry/backoff, timeout	Ném exception/exit khác 0; ghi kết quả theo contract
Lưu đầu ra phân tích	Giữ đường dẫn/summary nhỏ	Ghi Parquet; tạo metrics và manifest

Airflow task không đồng nghĩa Spark task. Trong DAG này, một task `detect_anomalies` của Airflow có thể kích hoạt nhiều Spark jobs, nhiều Spark stages và nhiều Spark tasks. Ngược lại, việc tăng `max_active_tasks` của Airflow không trực tiếp tăng số partition hay executor của Spark.

3.2 Vì sao không xử lý toàn bộ CSV bằng pandas trong PythonOperator?

Một PythonOperator chạy phép xử lý lớn trong môi trường Airflow có thể tiêu thụ CPU/RAM vốn cần cho scheduler và các task khác, khó tách dependency và khó mở rộng sang cluster. Nó cũng dễ khiến tác giả trả cả DataFrame vào XCom. Project dùng PythonOperator ở completion vì chỉ đọc JSON nhỏ, kiểm tra metadata và ghi log; đó là công việc phù hợp với orchestration.

Trong local mode, SparkSubmitOperator vẫn chạy compute trên cùng máy. Lợi ích đã có là ranh giới code/process, Spark DataFrame API, exit-code monitoring và khả năng thay backend khi đủ cấu hình. Không được diễn giải lựa chọn này thành “compute đã tách sang cluster độc lập”. Căn cứ nguyên tắc offload: sách [B8], chương 8 §8.3.3, trang in 185–186.

3.3 Vì sao chọn SparkSubmitOperator?

Operator thể hiện rõ mục đích submit Spark, nhận Connection ID, application path, arguments, driver memory và conf. SparkSubmitHook dựng lệnh, đọc output tiến trình và báo lỗi về Airflow. So với BashOperator chạy chuỗi shell, cấu hình có cấu trúc dễ đọc và giảm nhu cầu ghép lệnh thủ công. Operator không tự biến file local thành file có thể truy cập từ mọi worker cluster.

Project đặt `deploy_mode="client"`, `driver_memory="3g"`, `durable=False`. Trong chế độ local này không có remote driver để nối lại. Mỗi retry tạo một lần submit mới và ghi một generation mới; tính nhất quán dựa vào manifest, không dựa vào khôi phục một remote Spark application đã chạy dở. Đối chiếu provider [W3] và code DAG.

3.4 So sánh các mô hình tích hợp khi mở rộng

Mô hình	Ưu điểm	Điều kiện/chi phí	Hiện trạng
SparkSubmit + local[2]	Ít dịch vụ, dễ học, đủ dataset mẫu	Chung CPU/RAM, một máy	Đang dùng

Mô hình	Ưu điểm	Điều kiện/chi phí	Hiện trạng
SparkSubmit + YARN	ResourceManager cấp tài nguyên cluster	Cần client config, mạng, quyền, shared storage và log	Chưa triển khai
Spark trên Kubernetes	Driver/executor pods, cách ly tài nguyên	Image, service account/RBAC, namespace, volume/object storage	Chưa triển khai
DockerOperator/KubernetesPodOperator	Cô lập môi trường theo job	Quản lý image và volume; không tự có distributed Spark	Phương án thay thế
Dịch vụ quản lý như Databricks Jobs	Dùng API/connection và compute được quản lý	Tài khoản, chi phí, secret, job definition, mạng	Phương án mở rộng
SSH/Livy/HTTP submit	Tách nơi submit hoặc submit qua API	Theo dõi job, timeout, retry/idempotency phải rõ	Không có trong project

Đổi master sang YARN/Kubernetes không chỉ sửa chuỗi connection. Cần đảm bảo application và mọi đường dẫn input/output có thể được compute plane truy cập; cơ chế lock và `os.replace` local hiện tại cũng cần được thay bằng cơ chế commit phù hợp storage đích.

4. Các khái niệm Airflow được dùng trong bài

4.1 DAG, DagRun, Task và TaskInstance

DAG là đồ thị có hướng không chu trình mô tả các công việc và điều kiện phụ thuộc. File Python khai báo cấu trúc; scheduler không chạy toàn bộ nghiệp vụ chỉ vì file được import. `ecommerce_etl_dag` là định danh của định nghĩa DAG. Mỗi lần chạy tạo một DagRun, và mỗi task trong run có TaskInstance/trạng thái riêng.

`submit_pyspark_etl` là một Task trong định nghĩa; `submit_pyspark_etl` thuộc `official_smoke_20260929`, attempt 1 là một lần thực thi cụ thể. Retry tăng số lần thử của task; trigger DagRun mới tạo một định danh run khác. Phân biệt này giúp tìm đúng log và giải thích vì sao cùng một DAG có nhiều kết quả lịch sử.

4.2 Operator và Sensor

Operator là lớp mô tả cách làm một loại công việc. `EmptyOperator` tạo mốc luồng; `PythonOperator` gọi hàm Python; `SparkSubmitOperator` submit Spark. Khi khởi tạo operator với `task_id`, nó trở thành task trong DAG. Lựa chọn operator nên phản ánh trách nhiệm thay vì dùng một `PythonOperator` khổng lồ chứa mọi bước.

Sensor là operator chuyên chờ điều kiện. `LandingValidationSensor` kế thừa `FileSensor`: chọn đúng manifest, gọi vòng chờ của lớp cha, rồi mới kiểm tra contract và tạo context. Project mở rộng `execute` để bổ sung công việc sau khi ready, nhưng vẫn để `super().execute(context)` quản lý poke/reschedule/timeout. Không bắt `AirflowRescheduleException` rồi biến nó thành thành công. Đây là điểm cần kiểm thử riêng [B7, B9].

4.3 XCom, Params và Variables

XCom dùng chia sẻ giá trị nhỏ giữa task instances. Validation trả metadata; Spark task dùng Jinja lấy `context_path`. Không có `toPandas()` rồi push cả tập dữ liệu vào XCom. Cơ sở dữ liệu metadata không phù hợp lưu hàng trăm nghìn dòng giao dịch trong luồng điều phối.

Params là tham số có schema của DAG: `business_date`, `source_version`, `rfm_k` và `churn_days`. Người dùng nhập chúng khi trigger. Biên giá trị được kiểm tra bằng schema và helper. Variables là kho key-value cấu hình toàn cục của Airflow; project hiện không dùng Variable cho các tham số nghiệp vụ này. Nhờ đó ngày và ngưỡng được lưu theo run, tránh một Variable thay đổi làm lịch sử khó tái lập.

4.4 Connection và Hook

Connection đặt tên cho thông tin kết nối. Task tham chiếu `spark_default` hoặc `retail_landing`; thông tin thực tế được cung cấp bằng environment trong Compose. `spark_default` có `conn_type` spark và `host` `local[2]`. `retail_landing` có `conn_type` fs và `extra.path` trỏ tới landing root.

Hook là lớp client tích hợp hệ thống bên ngoài. `SparkSubmitHook` xử lý submit/theo dõi Spark; `FileSensor` dùng filesystem connection để tìm base path. Connection không phải chính tiến trình compute và Hook không phải lịch DAG. Định nghĩa tách ba lớp này giúp thay cấu hình môi trường mà không nhúng mọi đường dẫn kết nối vào từng task [B9].

4.5 Jinja và runtime context

`application_args` chứa biểu thức `{{ ti.xcom_pull(task_ids='validate_raw_data')['context_path'] }}`. Airflow render nó khi task chạy, sau khi validation thành công. Không gọi `xcom_pull` ở thời điểm import

DAG. Runtime context còn có `dag_run`, `params`, `run_id`, task instance và data interval nếu loại run có interval.

Hàm `business_date` xử lý trường hợp manual run không có logical date/data interval. Điều này phù hợp lần chạy thực tế: logical_date của manual run đã ghi nhận là rỗng, nhưng ngày nghiệp vụ vẫn rõ vì Params chứa D. Không dùng `datetime.now()` để suy ra ngày dữ liệu.

4.6 Scheduler, Executor, DAG processor và API server

Scheduler lập lịch, theo dõi state và đưa công việc vào executor khi đủ điều kiện. Executor là cơ chế chạy task Airflow; LocalExecutor chạy process local. DAG processor phân tích file DAG riêng. API server cung cấp giao diện/API và endpoint task execution của Airflow 3; FAB Auth Manager xử lý cơ chế xác thực được cấu hình.

Một file import lỗi có thể khiến DAG không xuất hiện hoặc bản mới không được cập nhật dù container vẫn healthy. Vì vậy cần kiểm tra cả `airflow dags list-import-errors`, trạng thái DagRun, trạng thái TaskInstance và nội dung output. Healthcheck chỉ chứng minh một phần khả năng sống của dịch vụ.

4.7 Retry, trigger rule và state

Các trạng thái quan trọng gồm `scheduled`, `queued`, `running`, `success`, `failed`, `up_for_retry`, `up_for_reschedule` và `upstream_failed`. Sensor chờ nguồn có thể reschedule mà chưa thất bại. Spark task exception có thể được retry. Task downstream phụ thuộc upstream thất bại không được chạy như thể dữ liệu đã sẵn sàng.

Completion dùng `all_success`, đòi hỏi cả RFM và audit thành công. Không dùng một leaf `all_done` luôn xanh để che thất bại nghiệp vụ. Callback thông báo lỗi không phải “sửa” trạng thái task hay tự công bố output thất bại [B6].

5. Docker và các thành phần triển khai

5.1 Image, container, build và Compose

Image là gói filesystem/dependency dùng khởi tạo container. Dockerfile kế thừa `apache/airflow:3.3.1`, chuyển sang root để cài Java 17, rồi trở về user airflow để pip install. `JAVA_HOME` trỏ tới `/usr/lib/jvm/java-17-openjdk-amd64`. Việc ghim lại `apache-airflow==${AIRFLOW_VERSION}` khi cài requirements giúp tránh resolver vô tình đổi Airflow sang phiên bản khác.

`docker compose build` tạo image; `docker compose up -d` tạo/chạy các container từ image đó. Sửa file bind mount có thể xuất hiện ngay trong container; sửa dependency trong requirements cần build lại image. Sửa environment/volume/connection trong Compose cần recreate service bằng `up -d`. `docker compose exec` chạy lệnh trong container đang hoạt động; `run --rm` tạo container một lần mới.

5.2 Vai trò từng service

Service	Command / cổng	Trách nhiệm và dấu hiệu bình thường
postgres	postgres:16; 5432 nội bộ	Metadata database; healthcheck pg_isready; dữ liệu trong named volume
airflow-init	version kèm cờ migrate/create-user	Chạy migration và khởi tạo tài khoản; kết thúc exit 0 là bình thường
airflow-apiserver	api-server; host 8080 → 8080	UI/API; healthcheck endpoint /api/v2/monitor/health
airflow-scheduler	scheduler	Lập lịch, LocalExecutor và Spark subprocess; healthcheck SchedulerJob
airflow-dag-processor	dag-processor	Phân tích DAG; healthcheck DagProcessorJob

Các service Airflow dùng chung YAML anchor `x-airflow-common`: image, user, environment và bind mounts. Anchor là cách tái sử dụng cấu hình YAML, không tạo thêm một container. `depends_on` với postgres healthy và init completed_successfully kiểm soát thứ tự khởi động ban đầu; không thay thế toàn bộ cơ chế phục hồi khi dịch vụ hỏng sau đó.

5.3 Volume, network và persistence

Host	Container	Chế độ và mục đích
./dags	/opt/airflow/dags	DAG và helper điều phối
./scripts	/opt/airflow/scripts	Job Spark và contract
./tests	/opt/airflow/tests	Read-only; chạy test trong đúng runtime
./data	/opt/airflow/data	Nguồn, landing, output, manifest
./logs	/opt/airflow/logs	Task logs và parser logs
./pyproject.toml	/opt/airflow/pyproject.toml	Read-only; cấu hình lint
postgres-db-volume	/var/lib/postgresql/data	Named volume giữ metadata PostgreSQL

Trong network Compose, service gọi nhau bằng tên DNS `postgres` hoặc `airflow-apiserver`. `localhost` bên trong scheduler là scheduler container, không phải Windows và cũng không phải PostgreSQL container. Người dùng Windows dùng `localhost:8080` vì có port mapping. Cổng 5432 không được publish ra host trong Compose hiện tại.

`docker compose down` dừng/xóa container và network nhưng giữ named volume mặc định. Thêm `--volumes` sẽ xóa metadata volume; đây không phải cách sửa lỗi thông thường. Bind-mounted data/log trên host không được xem là backup chỉ vì chúng còn tồn tại khi container bị thay thế.

5.4 Dependency và cấu hình Python

Thành phần	Phiên bản / nguồn	Vai trò
Apache Airflow	3.3.1 / base image	Orchestration runtime và SDK
Spark provider	6.3.2	SparkSubmitOperator / Hook
Standard provider	1.17.0	Empty/Python operators và FileSensor
FAB provider	3.8.0	FAB Auth Manager và quản trị user
PySpark	4.2.0	DataFrame, SQL, Window và Spark ML
Kaggle CLI	Không pin trong requirements.txt	Tải dataset; không phải task hằng ngày
pytest	8.4.2	Test runtime và Spark logic
Ruff	0.15.7 / requirements-dev.txt	Lint và định dạng code
Java	OpenJDK 17 JRE headless	Chạy JVM Spark trong image Linux
PostgreSQL	Image postgres:16	Airflow metadata database
Dependency tài liệu	requirements-docs.txt	Tạo Word/hình, đọc PDF; tách runtime

PySpark package cung cấp Python API và bộ thành phần Spark cần thiết cho cách chạy bằng image hiện tại; Java JVM là dependency riêng được Dockerfile cài. Provider Airflow cung cấp operator/hook, không phải tài nguyên cluster. `PYTHONPATH=/opt/airflow:/opt/airflow/scripts:/opt/airflow/dags` giúp DAG, sensor và Spark entry point tìm các module cùng project.

`requirements.txt` chưa phải lockfile hoàn chỉnh: Kaggle không được pin, dependency bắc cầu và image tag có thể thay đổi khi rebuild về sau. Bản chạy đã ghi lại các phiên bản chính. Muốn tái lập chặt hơn cần lưu constraints/lock và digest image; đó là cải tiến, không phải tính năng đã có.

5.5 Tài nguyên và cơ chế giới hạn

Mỗi Spark task có driver heap 3 GB và local[2]; JVM còn cần overhead, Python, bộ nhớ ngoài heap và page cache. Airflow, PostgreSQL và container khác cũng cần RAM. Vì vậy 3 GB heap không có nghĩa toàn job chỉ tốn đúng 3 GB RAM.

Project đặt `max_active_runs=1`, `max_active_tasks=2`, pool `retail_spark` một slot, mỗi Spark task dùng một slot. Hai nhánh có thể sẵn sàng đồng thời về phụ thuộc nhưng chỉ một JVM được cấp pool. Đây là lựa chọn bảo vệ tài nguyên đã kiểm chứng. Muốn chứng minh chạy song song thật phải đo đủ RAM, tăng pool lên hai và có Gantt/log overlap; bài hiện tại chưa có bằng chứng đó.

6. Bản đồ repository và trách nhiệm của từng phần

6.1 Cấu trúc module nghiệp vụ

`dags/ecommerce_etl_dag.py` chỉ khai báo DAG, Params, operator và dependency.

`dags/retail_support/tasks.py` chứa ngày nghiệp vụ, sensor, callbacks và completion.

`scripts/retail_contracts.py` chỉ dùng thư viện chuẩn để dùng chung giữa Airflow và Spark mà không kéo PySpark vào lúc parse DAG.

`scripts/retail_pipeline.py` điều phối công việc bên trong từng tiến trình Spark: đọc context, gọi transform legacy, tính metrics, viết Parquet và commit stage manifest. Ba file `pyspark_clean.py`, `pyspark_rfm.py`, `pyspark_anomalies.py` vẫn là entry points theo yêu cầu bài. Khi có `--pipeline-context`, chúng chuyển sang pipeline mode; nếu không có, CLI legacy hoạt động theo tham số riêng.

6.2 Thành phần nghiên cứu, kiểm thử và tài liệu

`select_kmeans_k.py` là công cụ offline đánh giá nhiều k; nó không nằm trong lịch DAG. Hai notebook EDA giải thích nguồn dữ liệu và quy tắc. Notebook là nơi pandas có thể phù hợp cho nghiên cứu ở quy mô mẫu; điều đó khác với đưa pandas compute lớn vào PythonOperator hằng ngày.

`tests/` chứa fixture và test cho cleaning, RFM, anomaly, contract, DAG, extensions Spark và cấu trúc repository. `docs/` chứa thiết kế, setup, báo cáo và bằng chứng.

`docs/evidence/official_run_20260929.json` là số liệu tổng hợp của run thật, không phải toàn bộ dữ liệu khách hàng.

6.3 Thành phần hỗ trợ cộng tác

`AGENTS.md` đặt quy tắc sửa repository; `CONTRIBUTING.md` hướng dẫn nhánh/commit/review; `.agent/TODO.md` ghi việc và người phụ trách; `HANDOFF` ghi trạng thái mới nhất; `DECISIONS` lưu quyết định lâu dài; `plans/active` và `plans/archive` quản lý kế hoạch. `.github/PULL_REQUEST_TEMPLATE.md` là mẫu PR. Các file này không phải task Airflow và không tham gia compute plane.

`.codex/config.toml` hiện là chỗ cấu hình hỗ trợ cộng tác, không chứa logic pipeline. `.codex/agents` và `.agents/skills` có placeholder. `.gitkeep` chỉ giữ thư mục trống trong Git. `.gitignore` loại environment, dữ liệu đầy đủ, output, logs và cache khỏi source control. File bị ignore vẫn tồn tại trên máy và vẫn có thể chứa dữ liệu cần bảo vệ.

6.4 Danh mục file và thư mục cần biết

Đường dẫn / nhóm	Trách nhiệm
<code>dags/ecommerce_etl_dag.py</code>	Định nghĩa sáu task, Params, date, concurrency và deadline
<code>dags/retail_support/tasks.py</code>	Sensor, ngày nghiệp vụ, completion và callbacks
<code>dags/retail_support/__init__.py</code>	Package marker cho helper DAG
<code>scripts/retail_contracts.py</code>	Contract file, hash, context, stage và publication
<code>scripts/prepare_daily_landing.py</code>	Bootstrap nguồn CSV bất biến theo ngày
<code>scripts/retail_pipeline.py</code>	Ba stage Spark, outputs/metrics/manifests
<code>scripts/pyspark_clean.py</code>	Cleaning legacy và entry ETL pipeline mode
<code>scripts/pyspark_rfm.py</code>	RFM, scores, KMeans và validator
<code>scripts/pyspark_anomalies.py</code>	Chín row rules, context và bảo toàn dòng

Đường dẫn / nhóm	Trách nhiệm
scripts/select_kmeans_k.py	Chọn k offline, CSV và chart tùy dependency
notebooks/EDA_Online_Retail.ipynb	Khảo sát nguồn/cleaning, không được DAG chạy
notebooks/EDA_Anomalies.ipynb	Nghiên cứu missing ID và quy tắc hồi cứu
notebooks/EDA_Anomalies_Guide_VI.md	Giải thích các bảng/biểu đồ EDA
tests/ và tests/fixtures/	Test logic, DAG, contracts và dữ liệu tổng hợp nhỏ
Dockerfile; docker-compose.yaml	Image và topology dịch vụ/volume/network
requirements.txt; requirements-dev.txt	Dependency chạy pipeline và lint
pyproject.toml	Ruff target Python/line length/rules/ngoại lệ legacy
.env; .env.example	Environment local / mẫu; không đưa secret vào báo cáo
data/raw/	CSV gốc, landing và manifest nguồn
data/curated/	Clean snapshots và audit input trung gian
data/analytics/rfm_daily/	Customer metrics và profile theo cutoff/version
data/analytics/k_selection/	Kết quả nghiên cứu số cụm offline
data/audit/anomalies/	Audit chính thức versioned
data/audit/anomalies.parquet/	Audit input legacy; khác root audit chính thức
data/audit/anomaly_results*/	Kết quả detector legacy và verification cũ
data/manifests/	Run contexts, stages, completion, published và locks
data/verification/	Artifacts kiểm chứng local, không phải output consumer
logs/	Log Airflow/Spark; không đưa toàn bộ vào Git
docs/REPORT.md; SETUP.md	Điều hướng báo cáo và hướng dẫn môi trường
docs/DAG_DESIGN_VI.md	Thiết kế có nguồn sách, traceability và acceptance plan
docs/IMPLEMENTATION_VI.md; evidence/	Hiện trạng và số liệu thực tế
docs/handbook/; docs/images/	Nguồn Word, hình minh họa và chỗ lưu hình
README.md; CONTRIBUTING.md; AGENTS.md	Giới thiệu và quy tắc cộng tác
.agent/	TODO, HANDOFF, DECISIONS, plans và quy ước trạng thái
.github/PULL_REQUEST_TEMPLATE.md	Mẫu nội dung review PR
.codex/; .agents/skills/	Cấu hình/placeholder hỗ trợ cộng tác, không phải runtime
.gitignore; .gitkeep	Kiểm soát file versioned và giữ thư mục trống

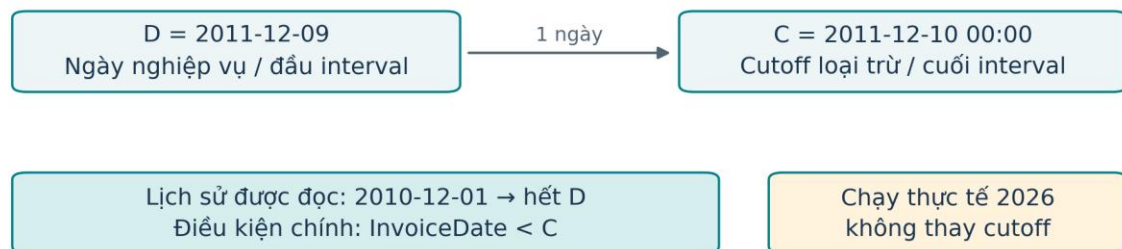
Không đọc `.env` hay `.env.example` có giá trị thực để làm tài liệu. Mẫu cấu hình an toàn ở chương cài đặt chỉ liệt kê tên biến và cách tự tạo giá trị. Thư mục `docs/handbook` cùng script dựng tài liệu thuộc lớp documentation, không được import vào DAG.

7. Ngày nghiệp vụ, lịch chạy và backfill

7.1 Bốn khái niệm ngày khác nhau

Khái niệm	Trong project	Ví dụ
Thời điểm giao dịch	InvoiceDate trong nguồn	2011-12-09 12:50
Ngày nghiệp vụ D	Ngày cuối của dữ liệu được chọn	2011-12-09
Cutoff C	D + một ngày, loại trừ ở biên trên	InvoiceDate < 2011-12-10 00:00
Thời điểm chạy thực	Máy thực thi job trong hiện tại	29/09/2026

Ngày hiện tại không được dùng thay cutoff. Nếu dùng năm 2026 để tính Recency trên dữ liệu kết thúc năm 2011 thì gần như mọi khách đều bị coi là lâu không mua, làm mất ý nghĩa demo. Ngược lại, chạy lại lịch sử ngày hôm nay vẫn phải cho ra kết quả as-of của cutoff lịch sử [B3, B5].



Hình 3. Phân biệt data interval, ngày nghiệp vụ và thời điểm chạy thực.

7.2 Timetable và giới hạn lịch sử

DAG dùng CronDataIntervalTimetable với `0 0 * * *`, timezone UTC, start_date 2010-12-01 và end_date 2011-12-09. Interval cuối hợp lệ bắt đầu 2011-12-09 và kết thúc 2011-12-10. End date giới hạn điểm bắt đầu interval được tạo theo timetable; không có nghĩa loại bỏ mọi giao dịch trong ngày 09/12.

`catchup=False` không tự tạo toàn bộ khoảng lịch sử khi bật DAG. Đó là lựa chọn để tránh hàng trăm full-history snapshots đồng loạt trên laptop. `is_paused_upon_creation=True` chỉ quy định trạng thái khi DAG mới được tạo trong metadata; nó không ép một DAG đã tồn tại tự pause lại sau mọi lần reload.

Nguồn timestamp không có timezone. Project thống nhất session và timetable UTC để cắt ngày nhất quán trong demo; không có quy trình xác minh hoặc chuyển đổi lịch sử múi giờ kinh doanh/DST của nguồn. Nếu doanh nghiệp cần ngày địa phương, phải chốt quy tắc timezone trước khi thay đổi contract.

7.3 Manual, scheduled và backfill

Manual run yêu cầu business_date rõ ràng. Scheduled/backfill lấy D từ data_interval_start và kiểm tra interval đúng một ngày từ nửa đêm. Nếu Params chứa business_date khác interval, validation từ chối. Điều này ngăn một đợt backfill nhiều ngày vô tình dùng chung cutoff do cấu hình override.

Backfill tạo các run cho interval lịch sử; retry chạy lại task của cùng run; clear xóa state thực thi để task được xét chạy lại; trigger manual tạo run mới theo Params. Bốn thao tác phục vụ mục đích khác nhau. Trong dự án, thay đổi source/code/params đòi hỏi DagRun mới vì context của run cũ đã bị đóng băng.

7.4 Vì sao snapshot tích lũy?

Mỗi ngày D đọc lịch sử từ ngày nguồn đầu tiên đến D, kèm undated, rồi tính lại snapshot. RFM cần lịch sử mua trước cutoff, không thể chỉ tính từ file của ngày D. Vì snapshot độc lập, chạy ngày B trước ngày A vẫn có thể đúng, không cần DAG ngày A đã thành công. `depends_on_past=False` phù hợp lựa chọn đó.

Đổi lại, nhiều cutoff lặp lại chi phí đọc/ghi. Đây chưa phải incremental update, CDC hay exactly-once processing. Với quy mô lớn nên tách fact table đã chuẩn hóa, checkpoint/watermark, merge/upsert và recomputation có phạm vi, nhưng phải giữ hợp đồng late data và lịch sử phiên bản rõ ràng.

7.5 Quy trình backfill thận trọng

Đọc help trong đúng image, preview khoảng một hoặc hai ngày, xác nhận source coverage, pool và dung lượng đĩa. Không chạy cả năm chỉ để thử lệnh. Không truyền `business_date` cố định khi tạo nhiều interval. Sau khi hoàn tất, kiểm tra riêng mỗi published cutoff, không chỉ đếm số DagRun success.

```
docker compose exec airflow-scheduler airflow backfill create --help
docker compose exec airflow-scheduler airflow backfill create --dag-id ecommerce_etl_dag --from-date 2011-12-08 --to-date 2011-12-09 --reprocess-behavior none --max-active-runs 1 --dry-run
```

Cú pháp trên đã đối chiếu help Airflow 3.3.1 trong image. Bỏ `--dry-run` là thao tác tạo backfill thật, chỉ làm sau khi xem preview. Bản kiểm chứng hiện có chưa thực thi chiến dịch backfill nhiều ngày; unit test đã kiểm tra biên interval và việc từ chối override sai.

8. Thiết kế chi tiết sáu task

8.1 Bảng contract của từng task

Task	Đầu vào	Công việc chính	Đầu ra / điều kiện thành công
start_pipeline	DagRun đã được scheduler xét	Mốc bắt đầu bằng EmptyOperator	State success; không tạo dữ liệu
validate_raw_data	Params, interval, source manifest	Chờ nguồn; kiểm tra coverage/header/size; cố định context	context.json và XCom nhỏ
submit_pyspark_etl	context_path	SHA-256, schema, dedup, cleaning, quarantine	transactions, audit_input, parse_errors; etl.json
compute_rfm_metrics	Cùng context và etl.json	RFM, score, KMeans, business labels	customers, cluster_profile; rfm.json
detect_anomalies	Cùng context và etl.json	Chín row checks và invoice threshold	row_assessments, order_assessments, flagged_orders; audit.json
notify_completion	Ba stage manifests	Đối chiếu generation; ghi completion/published; log	PIPELINE_COMPLETE, con trở published

8.2 start_pipeline và validate_raw_data

EmptyOperator không chạy ETL và không cần đọc file. Nó làm sơ đồ và điểm bắt đầu rõ theo đề.

Validation có mode reschedule, poke mỗi 60 giây, tổng thời gian chờ 1.800 giây, `soft_fail=False`, `deferrable=False` và `retries=0`. Khi chưa có manifest, nó nhường worker giữa các lần poke; project không cần triggerer cho lựa chọn này.

Sau khi manifest tồn tại, sensor kiểm tra version/schema, ngày yêu cầu nằm trong coverage, mọi partition lịch sử phải có entry, đường dẫn không thoát root, file tồn tại, kích thước lớn hơn 0 và khớp bytes đã công bố, header chính xác tám cột. File chỉ có header hợp lệ biểu diễn ngày không có giao dịch; file 0 byte là lỗi. Chưa có manifest là “chưa ready”; manifest complete nhưng thiếu file là “contract hỏng”, không phải lý do chờ vô hạn.

Validation tạo run_key từ dag_id và run_id, ghi context bằng atomic replace và trả metadata qua XCom. Nếu context cùng run đã tồn tại nhưng tham số/source/code khác thì fail. Khối tạo context không chạy khi FileSensor còn reschedule. Những hành vi này có unit tests riêng.

8.3 Ba SparkSubmitOperator

Helper `spark_job` giữ cấu hình chung: application dưới `/opt/airflow/scripts`, một argument `--pipeline-context`, Connection spark_default, client mode, 3g driver, session UTC, pool retail_spark, pool_slots=1 và `do_xcom_push=False`. Ba task khác nhau ở script và hard timeout: ETL 20 phút, RFM 20 phút, audit 30 phút.

ETL phải thành công trước cả hai nhánh. Nhánh RFM và audit không phụ thuộc nhau, nhưng cùng đọc stage manifest ETL đã commit. Khi pool chỉ có một slot, scheduler chọn một nhánh chạy trước; không được viết logic cho rằng RFM luôn phải xong rồi audit mới đúng về nghiệp vụ. Lần kiểm chứng đã chạy RFM trước, nhưng đó là thứ tự thực thi được ghi nhận, không phải dependency bổ sung.

8.4 notify_completion

PythonOperator lấy metadata validation qua XCom, load context và xác minh code version. Nó load ba stage manifests, kiểm tra context hash, complete flag, các `_SUCCESS` marker và ETL hash của hai analytics branches. Chỉ sau đó mới lấy lock theo cutoff, ghi completion.json của run và thay con trỏ published của cutoff.

Hàm này không gửi email/Slack và không xử lý DataFrame. Đề cho phép log summary hoặc gửi thông báo; project chọn log `PIPELINE_COMPLETE` cùng metrics/đường dẫn vì chưa cấu hình người nhận hay dịch vụ gửi. Callback lỗi/retry/deadline cũng chỉ ghi log.

8.5 Retry và các mốc thời gian

Cấu hình	Giá trị	Giải thích
owner	retail-data-team	Metadata người phụ trách, không phải tài khoản gửi email
retries mặc định	1	Tối đa một lần thử lại sau lần đầu nếu lỗi cho phép retry
retry_delay	5 phút	Khoảng chờ cơ sở; không phải thời gian chạy job
exponential_backoff / max_retry_delay	True / 15 phút	Backoff có giới hạn; không coi mọi lần retry chính xác 5 phút
start, validation retries	0	Không retry mốc rỗng; validation hỏng contract cần sửa nguồn
validation execution_timeout	2 phút	Giới hạn thực thi, tách khỏi tổng thời gian sensor chờ
sensor timeout	30 phút	Thời gian chờ readiness; timeout dẫn đến failed
task timeout	1 / 2 / 20 / 20 / 30 / 2 phút	Lần lượt sáu task theo thứ tự và hai nhánh
dagrun_timeout	90 phút	Giới hạn tổng runtime của DagRun theo Airflow
deadline	queued_at + 60 phút	Gọi SyncCallback nếu run quá hạn; không tự làm task success

8.6 SLA, DeadlineAlert và freshness

Airflow 3.3.1 sử dụng DeadlineAlert trong DAG này; không chép tham số SLA cũ từ ví dụ Airflow 2. Callback đồng bộ được executor thực thi và nhận context giới hạn. Khai báo hiện tại dùng `DAGRUN_QUEUED_AT` để việc chạy lại ngày lịch sử không bị coi trễ nhiều năm ngay lập tức. API deadline được tài liệu chính thức đánh dấu experimental [W1].

Deadline 60 phút là mục tiêu giám sát của demo; timeout là cơ chế giới hạn thực thi. Freshness lại là câu hỏi khác: kết quả cho ngày D có được công bố đúng thời hạn phục vụ người dùng hay không. Một scheduler chết trước khi tạo run không được callback của run chưa tồn tại phát hiện. Chưa có external watchdog trong project. Đã kiểm chứng import/khai báo; chưa cố tình chạy quá 60 phút để chứng minh alert thực tế.

9. Dữ liệu nguồn và landing bất biến

9.1 Dataset và schema

Dataset chọn là Kaggle [carrie1/ecommerce-data](#), bản giao dịch Online Retail của UCI. CSV đang dùng có 541.909 dòng, tám cột, khoảng ngày 01/12/2010–09/12/2011. Nguồn là dữ liệu lịch sử; giả định doanh nghiệp “millions of daily records” trong đề là bối cảnh kiến trúc, không phải kích thước đã benchmark.

Cột	Kiểu nghiệp vụ sau parse	Ý nghĩa / chính sách
InvoiceNo	string	Mã hóa đơn; tiền tố C được dùng nhận diện hủy
StockCode	string	Mã sản phẩm hoặc mã phí/điều chỉnh; giữ ký tự
Description	string	Mô tả; không tự suy diễn mọi từ khóa thành lỗi
Quantity	integer 32-bit	Số lượng; giá trị âm/0 được giữ trong audit
InvoiceDate	timestamp	Parse mẫu M/d/yyyy H:mm; dùng cutoff exclusive
UnitPrice	double	Đơn giá; kiểm tra hữu hạn và dấu theo từng luồng
CustomerID	string	Định danh, không phải thước đo số; thiếu không được gộp thành một khách giả
Country	string	Thuộc tính địa lý; kiểm tra nhất quán cấp hóa đơn

Currency của dataset là sterling/GBP [W6]. Project không có bảng tỷ giá, không quy đổi sang USD và không áp số \$5.000 như thể đó là cùng đơn vị với nguồn. Đề đưa các ví dụ ngưỡng; implementation chọn ngưỡng thống kê phù hợp đơn vị đã biết.

9.2 Bootstrap làm những gì?

`prepare_daily_landing.py` chạy ngoài DAG, phục vụ chuẩn bị bản nguồn lịch sử bất biến. Nó đọc đúng tám trường CSV bằng thư viện `csv`, giải mã CP1252, nhóm theo ngày `InvoiceDate`, đưa ngày không parse được vào `undated`, tạo UTF-8 CSV cho mọi ngày từ `first_date` đến `last_date` và đọc lại để đối chiếu chính xác từng giá trị chuỗi.

Những record có số trường khác tám hoặc header sai khiến bootstrap fail thay vì tự cắt/bù cột. Sai định dạng thời gian có nơi giữ riêng `undated`; sai số liệu sẽ được Spark parse/quarantine sau. Thư viện `csv` xử lý dấu phẩy, dấu ngoặc kép và newline trong trường; không dùng `split(',')` vốn làm hỏng mô tả sản phẩm.

Bootstrap giữ các hàng trong RAM để phân nhóm; đây là công cụ một lần cho dataset khoảng 0,5 triệu dòng, chưa phải ingestion streaming có bộ nhớ bị chặn. Tên source version mặc định `online-retail-v1` là nhãn; SHA-256 trong manifest mới xác nhận nội dung. Chạy lại cùng nguồn/encoding trả manifest hiện có. Nguồn khác phải chọn version mới.

9.3 Bố cục landing

```
data/raw/data.csv
data/raw/landing/online-retail-v1/
  manifest.json
  date=2010-12-01/data.csv
  ...
  date=2011-12-09/data.csv
  undated/data.csv
```

Ngày không có giao dịch vẫn có file header-only và `rows=0`. Điều này cho phép phân biệt “nguồn xác nhận ngày trống” với “producer chưa gửi file”. Manifest chứa `schema_version`, `source_version`, `source_sha256`,

source_encoding, parser, first_date/last_date, tổng rows, entry undated và partitions theo ngày. Mỗi entry có path tương đối, bytes, sha256 và rows.

Ghi staging trong thư mục ẩn mới; sau khi mọi ngày round-trip hợp lệ, checksum nguồn không đổi và manifest hoàn chỉnh, bootstrap rename sang tên version chính thức. Nếu dở dang, không giả vờ rằng version đã sẵn sàng. Không tự xóa thư mục dở của người dùng khi retry.

9.4 Các lớp bảo vệ nguồn

Sensor kiểm tra metadata/file nhẹ trước khi khởi động Spark. ETL kiểm tra đầy đủ hash từng file trong phạm vi, đối chiếu source manifest hash với context, đối chiếu số dòng đọc với tổng entry rows, và từ chối transaction có ngày tương lai so với cutoff dù file bị đặt nhầm partition.

SHA-256 phát hiện thay đổi nội dung, kể cả thay đổi giữ nguyên kích thước file. Nó không tự chứng minh tính đúng nghiệp vụ hoặc nguồn được tin cậy về mặt pháp lý. Hệ thống local giả định publisher và quyền ghi nguồn được kiểm soát; đây không phải kho object immutable có access policy/WORM.

9.5 Dòng undated và giới hạn thời gian

Dòng không có ngày hợp lệ không thể đặt vào một ngày nghiệp vụ chắc chắn. Pipeline giữ chúng trong audit và quarantine khi phù hợp, không đưa vào RFM. Undated của snapshot nguồn được đưa vào mọi cutoff được đọc; do không biết ngày thật, không thể khẳng định chúng là sự kiện đã xảy ra trước D. Các phép kiểm tra cần thời gian sẽ ghi không áp dụng. Dữ liệu thật đã chạy có zero parse-error rows; chính sách undated được kiểm thử bằng fixture.

10. PySpark ETL và Partitioned Parquet

10.1 Parse tường minh, không schema inference

`read_landing` đọc UTF-8 với schema tám STRING, `header=True`, `mode=FAILFAST`, `multiLine=True` và escape dấu ngoặc kép. Sau đó biểu thức Spark chuyển Quantity bằng `try_cast INT`, UnitPrice bằng `try_cast DOUBLE` và InvoiceDate bằng `try_to_timestamp`. ID và mô tả giữ dạng chuỗi; literal rỗng/NaN được xử lý theo biểu thức của từng cột.

Đầu ra parse gồm typed transactions và rejected rows có cả tám `raw_*` fields lẫn cột đã chuyển kiểu. rejected chỉ đánh dấu giá trị số/ngày không rỗng nhưng chuyển thành null. NaN/Infinity parse được sang double có thể không nằm trong `parse_errors`; chúng vẫn bị xử lý bởi kiểm tra finite/quality ở bước nghiệp vụ. Vì vậy “`parse_errors=0`” không có nghĩa nguồn không có null, giá âm hoặc giao dịch cần rà soát.

10.2 Dedup và hai chính sách giữ dữ liệu

`prepare_datasets` dropDuplicates trên toàn bộ tám cột, rồi tách hai DataFrame. Audit input giữ các dòng duy nhất, bao gồm null, số âm, số 0, hủy và phí. RFM input yêu cầu các cột không null, hóa đơn không bắt đầu C sau trim/upper, Quantity>0, UnitPrice>0 và loại các mã/dòng phí đã chọn. Pipeline mode bổ sung UnitPrice hữu hạn và InvoiceNo/CustomerID không blank.

Dedup toàn dòng loại các bản sao giống hệt; nó không chứng minh mọi business duplicate đã được loại và không giải quyết hai dòng cùng khóa nhưng khác giá/số lượng. Không có nghiệp vụ suy ra rằng mỗi InvoiceNo–StockCode chỉ được xuất hiện một lần. Bổ sung một quy tắc dedup khác cần yêu cầu nghiệp vụ và test riêng.

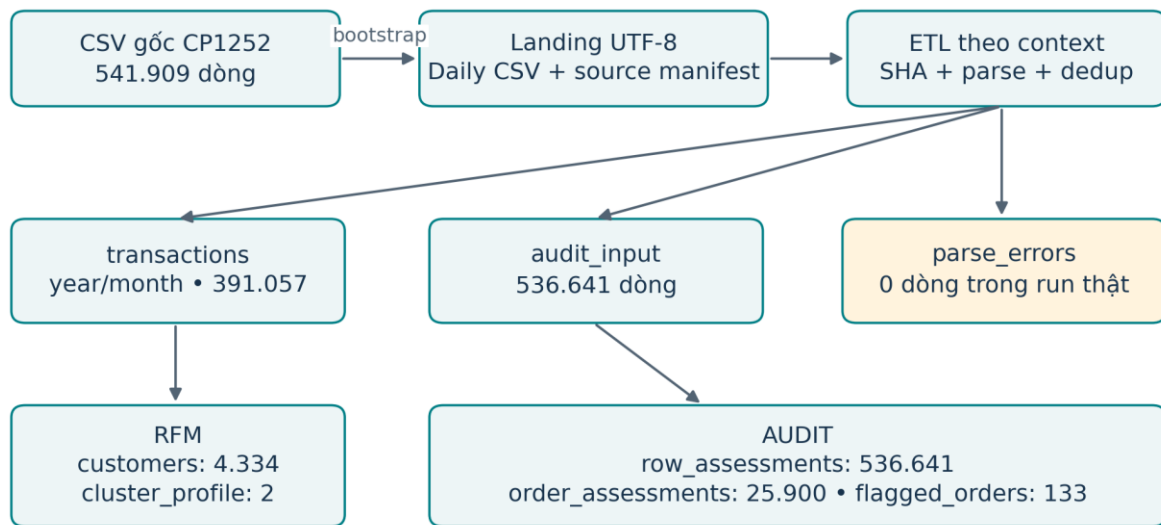
Trường hợp	RFM input	Audit input	Lý do
CustomerID null	Loại	Giữ	Không tổng hợp được theo khách; vẫn rà soát dòng/đơn được
InvoiceNo bắt đầu C	Loại	Giữ	Không tính như mua hàng dương; cần kiểm tra hoàn/hủy
Quantity hoặc UnitPrice <= 0	Loại	Giữ	RFM định nghĩa trên giao dịch dương; audit cần thấy giá trị gốc
Giá không hữu hạn	Loại	Giữ với DQ flag	Tránh làm hỏng tiền và ngưỡng
Trùng toàn bộ record	Giữ một bản	Giữ một bản	Không đếm đôi cùng record nguồn
Mã phí hoặc mô tả phí đã chọn	Loại	Giữ	Không trộn chi tiêu sản phẩm với phí trong RFM
Ngày không parse được	Loại	Giữ theo phạm vi kiểm tra	Không thể tính Recency hoặc gán tháng đúng

Danh sách StockCode loại riêng khỏi RFM: POST, M, DOT, BANK CHARGES, C2, PADS. Hai mô tả exact match là PACKING CHARGE và NEXT DAY CARRIAGE, so không phân biệt hoa thường và bỏ khoảng trắng hai đầu. Một sản phẩm có chữ “carriage” ở giữa tên không tự bị loại.

10.3 Ghi dữ liệu và ý nghĩa partition

Clean transactions thêm year, month từ InvoiceDate rồi ghi Parquet Snappy theo `partitionBy("year", "month")`. Output nằm dưới cutoff và generation của run. Ví dụ đường dẫn đầy đủ có dạng `data/curated/run_date=2011-12-10/version=.../transactions/year=2011/month=12/`. Đề yêu cầu phân vùng year/month; prefix run/version là phần bổ sung để tránh xung đột rerun, không đổi nội dung yêu cầu phân vùng.

Parquet là định dạng cột: giữ schema, đọc cột cần thiết và hỗ trợ predicate/partition pruning. Một thư mục dataset chứa nhiều part files; `.parquet` có thể là tên thư mục trong chế độ legacy. Không mặc định gom tất cả thành một file bằng `coalesce(1)`, vì như vậy tạo bottleneck một task ghi.



Mỗi stage ghi manifest sau khi đủ output; completion mới công bố cả tập kết quả.

Hình 4. Luồng dữ liệu và tám output Parquet của pipeline chính thức.

10.4 Spark execution và hiệu năng

Transformation như filter/select/groupBy tạo kế hoạch lazy; action như count, collect nhỏ, isEmpty và write mới thực thi. Shuffle xảy ra ở dedup, grouping, join và Window. Job đặt shuffle partitions=8 cho demo; local[2] chỉ thực thi số task phù hợp tài nguyên tại một thời điểm. Tám partitions không đồng nghĩa tám CPU cores.

Cache được dùng cho dữ liệu phải kiểm tra và ghi nhiều lần; unpersist nằm trong finally để giải phóng tài nguyên. SparkSession cũng được stop trong finally. Code không collect toàn bộ transactions vào driver; chỉ collect các bảng thống kê nhỏ hoặc một hàng tham chiếu. Các lựa chọn này không loại bỏ hoàn toàn recomputation hay nhu cầu tuning ở dataset lớn.

10.5 Các đối soát ETL đã quan sát

541.909 dòng nguồn = 536.641 dòng sau dedup + 5.268 dòng trùng. Trong 536.641 dòng duy nhất, 391.057 dòng vào RFM và 145.584 dòng bị loại khỏi nhánh RFM. Toàn bộ 536.641 dòng vẫn là audit input. Có zero parse errors trong run thật. Đọc lại Parquet xác nhận 13 tổ hợp year/month, từ tháng 12/2010 đến tháng 12/2011.

Các số trên thuộc pipeline parser UTF-8 đã kiểm chứng. Output/notebook legacy có thể có lịch sử parser/encoding hoặc rule khác; không lấy số cũ thay cho bằng chứng run chính thức. Trước khi so sánh, phải kiểm tra source, cutoff, rule version và code version.

11. RFM, phân khúc và inactivity proxy

11.1 Định nghĩa toán học

Với khách hàng u , gọi $H(u,C)$ là tập giao dịch clean có $\text{InvoiceDate} < C$. LastPurchaseDate là thời điểm mua lớn nhất trong tập đó. Frequency là số InvoiceNo phân biệt, không phải số dòng sản phẩm. Monetary là tổng $\text{Quantity} \times \text{UnitPrice}$ trên các dòng được giữ. Recency là số ngày từ ngày mua gần nhất đến cutoff.

```
LastPurchaseDate(u,C) = max(InvoiceDate trong H(u,C))
Recency(u,C) = datediff(C, date(LastPurchaseDate(u,C)))
Frequency(u,C) = countDistinct(InvoiceNo trong H(u,C))
Monetary(u,C) = round(sum(Quantity * UnitPrice), 2)
```

Quantity và UnitPrice được cast decimal(18,4) khi nhân/tổng trong RFM để hạn chế sai số tiền; đơn giá gốc vẫn lưu double trong transaction schema. Nếu miền tiền lớn vượt khả năng decimal hiện tại, job cần schema và test mới; không coi fixed precision là không có giới hạn.

11.2 Ví dụ tính bằng tay

Giả sử khách A có hai dòng cùng hóa đơn INV1 ngày 08/12: 2×10 và 1×5 GBP, và một dòng INV2 ngày 09/12: 3×7 GBP. Với cutoff 10/12/2011, Recency=1, Frequency=2, Monetary=46 GBP. Đếm số dòng sẽ cho Frequency=3 và sai định nghĩa của bài.

Vì cutoff là đầu ngày kế tiếp và chỉ nhận giao dịch trước cutoff, khách mua ngày D có Recency=1. Đây không phải lỗi off-by-one nếu nhất quán với contract. Monetary hiện phản ánh tổng mua dương sau loại hủy/âm/phí; không phải doanh thu thuần kế toán đã bù hoàn tiền.

11.3 Tính điểm R/F/M bằng Window

Code dùng percent_rank trên từng thước đo. Với Recency, thứ tự giảm dần làm khách lâu không mua có rank thấp và điểm thấp. Frequency/Monetary sắp tăng dần: giá trị lớn có điểm cao. Các khoảng rank [0;0,2), [0,2;0,4), [0,4;0,6), [0,6;0,8), [0,8;1] ánh xạ điểm 1–5.

Các giá trị bằng nhau được giữ cùng rank; CustomerID không được thêm vào ORDER BY để phá hòa và làm những khách có giá trị bằng nhau nhận điểm khác nhau. Nhóm có thể không đều và thiếu một số mức điểm. Một khách duy nhất hoặc một thước đo hằng nhận điểm 1 theo implementation, không tự mặc định điểm trung tính 3.

RFM_score là chuỗi nối ba điểm, ví dụ 545. Đây không phải tổng 5+4+5 và không phải xác suất. Điểm là tương đối so với tập khách trong cutoff đó; cùng một khách có thể đổi điểm dù Monetary không đổi nếu quần thể tham chiếu đổi.

11.4 KMeans và lựa chọn k

Các feature Recency, Frequency, Monetary được log1p để giảm độ lệch phải, VectorAssembler gộp thành vector, StandardScaler chuẩn hóa withMean=True/withStd=True, rồi KMeans fit với seed 42. Code kiểm tra $k \geq 2$, k không vượt số khách, đủ số vector phân biệt và có đúng k cụm thực sự được chiếm.

Script offline [select_kmeans_k.py](#) dùng cùng pipeline feature, fit scaler một lần khi so các k, ghi WSSSE và silhouette vào CSV; biểu đồ được tạo nếu matplotlib có sẵn. WSSSE thường giảm khi k tăng nên không dùng “WSSSE nhỏ nhất” để tự chọn số cụm tối đa. Silhouette đánh giá cấu trúc phân tách trong không gian feature, không chứng minh nhóm có giá trị marketing.

Artifact nghiên cứu hiện có ghi silhouette k=2 khoảng 0,6238; k=4 khoảng 0,4918. DAG mặc định k=2 dựa trên bằng chứng nghiên cứu đó và có Params để đổi. Đây là lựa chọn demo cho nguồn/cutoff đã xem; chưa chạy lại đánh giá k trên mọi snapshot và không tuyên bố k=2 tối ưu cho mọi ngày.

Cluster ID chỉ là nhãn thuật toán. Không suy ra Cluster 0 luôn là VIP. Output profile hiện có số khách và mean R/F/M theo cụm; chưa lưu model object, centroids hay median đầy đủ. Phân khúc nghiệp vụ dưới đây không phụ thuộc số ID cụm.

11.5 Quy tắc nhãn khách hàng

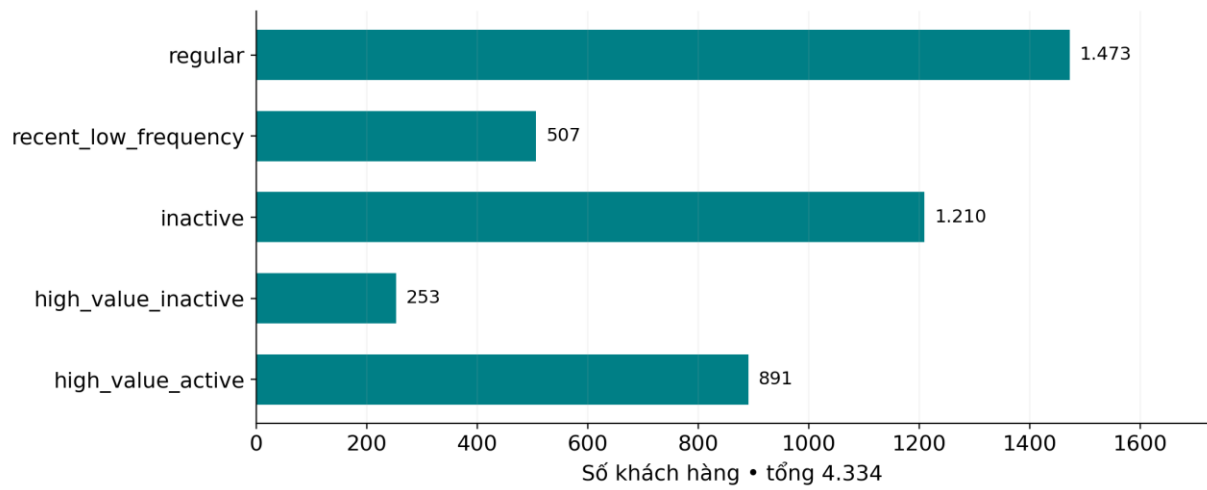
Trường / nhãn	Điều kiện	Ý nghĩa
is_high_value	M_score >= 4	Giá trị tiền tương đối cao trong quần thể
is_inactive	Recency >= churn_days	Không mua trong ít nhất ngưỡng ngày đã chọn
high_value_inactive	High-value và inactive	Ưu tiên trước các nhãn khác
high_value_active	High-value, F_score >= 4 và R_score >= 4	Khách tốt theo cả ba chiều
inactive	Inactive, chưa rơi vào nhãn ưu tiên	Khách lâu không mua
recent_low_frequency	R_score >= 4 và F_score <= 2	Mua gần đây nhưng tần suất tương đối thấp
regular	Các trường hợp còn lại	Nhãn còn lại theo quy tắc hiện hành

Thứ tự when/otherwise bảo đảm mỗi khách có đúng một segment. Các boolean high-value/inactive có thể giao nhau; tổng hai boolean không được coi là số khách duy nhất. Ví dụ 253 khách của run thật đồng thời high-value và inactive.

11.6 Churn risk là proxy có điều kiện

Ngưỡng mặc định là 90 ngày, có thể đổi từ 1 đến 3.650 qua Params. Code lưu churn_threshold_days, observed_history_days và segment_rule_version. Nếu thời gian nguồn quan sát ngắn hơn churn_days, churn_assessment_status=insufficient_history; nếu đủ lịch sử, inactive nhận inactivity_proxy, còn lại not_flagged.

Boolean is_inactive vẫn là phép so Recency với ngưỡng; trạng thái assessment mới cho biết mức đủ thông tin. Khi diễn giải kết quả, phải xem cả hai. Project chưa có nhãn churn tương lai, mô hình dự báo xác suất, train/test thời gian hay đánh giá AUC. Không đưa tỷ lệ 1.463/4.334 vào báo cáo như “tỷ lệ khách chắc chắn rời bỏ”.



Hình 5. Số khách theo nhãn nghiệp vụ trong run đã kiểm chứng.

12. Chất lượng dữ liệu và phát hiện bất thường

12.1 Vì sao cần hai cấp audit?

Audit dòng kiểm tra lỗi/chênh lệch ở Quantity, UnitPrice, line value và ngữ cảnh hóa đơn. Audit tổng đơn gom các dòng theo InvoiceNo để đánh giá tổng tiền. Hai dòng không vượt IQR riêng lẻ vẫn có thể nằm trong một đơn rất lớn. Ngược lại một đơn có tổng bình thường vẫn có thể chứa một dòng giá âm. Vì vậy hai loại đầu ra bổ sung nhau và không được cộng trực tiếp số lượng cờ.

Nhánh audit đọc 536.641 dòng đã dedup, không đọc 391.057 dòng RFM-only. Nếu đọc nhánh đã loại hết null/âm/hủy thì detector mất các trường hợp cần kiểm tra. Missing CustomerID tự nó không phải bằng chứng gian lận và không làm mọi phép kiểm tra mất khả năng áp dụng.

12.2 Ba nhóm thông tin trên mỗi dòng

Nhóm	Ví dụ	Có tự biến thành nhãn gian lận không?
data_quality_flags	missing_customer_id, missing_invoice_date, invalid_unit_price	Không; mô tả dữ liệu thiếu/không hợp lệ
context_flags	cancelled_invoice, service_line, keyword_product_name	Không; cung cấp bối cảnh nghiệp vụ
check_results / anomaly_flags	Kết quả từng rule và danh sách rule flagged	Là ứng viên rà soát theo quy tắc, không phải nhãn đã xác thực

Các DQ flags còn có missing_invoice_no, missing_stock_code, missing_description, missing_country, invalid_quantity và invalid_line_value. Giá trị số âm có thể hợp lệ về kiểu dữ liệu; nó chỉ vi phạm một số rule/ngữ cảnh cụ thể, khác với NaN hoặc Infinity.

12.3 Sáu quy tắc nghiệp vụ trên dòng

Rule	Điều kiện gắn cờ	Ví dụ trường hợp không áp dụng
cancel_nonnegative_quantity	Invoice C nhưng Quantity ≥ 0	Thiếu invoice hoặc quantity không hợp lệ
negative_quantity_non_cancel	Quantity < 0 nhưng invoice không C	Thiếu invoice/quantity
negative_unit_price	UnitPrice < 0	Giá không hữu hạn hoặc không hợp lệ
zero_price_in_priced_invoice	Dòng giá 0 trong invoice có dòng giá dương	Không đủ giá để kết luận và chưa thấy dòng dương
all_zero_price_invoice	Mọi dòng invoice có giá hợp lệ và đều 0	Invoice có giá thiếu/không hợp lệ
multiple_customer_ids	Invoice có ít nhất hai CustomerID khác nhau	Chưa thấy nhiều ID nhưng còn dòng thiếu ID

Hai điểm tinh tế: một invoice đã quan sát hai ID thì có thể gắn cờ dù còn ID thiếu; ngược lại chỉ quan sát một ID kèm null chưa đủ kết luận toàn invoice chỉ có một khách. Tương tự, đã thấy một dòng giá dương có thể chứng minh bối cảnh “priced invoice” ngay cả khi một số dòng khác chưa có giá.

12.4 Ba quy tắc IQR theo sản phẩm

Tập tham chiếu gồm dòng có ngày, invoice, StockCode, Quantity>0, UnitPrice hữu hạn >0, line_value>0, không hủy, không thuộc phí/mã đặc biệt. Nhóm có/không có CustomerID cùng tham gia nếu đáp ứng các điều kiện còn lại. Mỗi StockCode cần ít nhất 30 dòng đủ điều kiện; IQR phải dương.

```
IQR = Q3 - Q1
UpperBound = Q3 + 3 * IQR
flagged khi observed_value > UpperBound
```

Ba observed values là Quantity, UnitPrice và Quantity×UnitPrice. Spark dùng percentile chính xác ở Q1/Q3 theo implementation, không fallback sang ngưỡng toàn cục khi sản phẩm ít mẫu. Nếu n<30, IQR=0, giá trị không hữu hạn hoặc ngoài nhóm bán hàng, check nhận not_applied cùng reason, không mặc định not_flagged.

Reference dùng toàn bộ lịch sử trước cutoff cùng với các dòng được chấm, nên đây là phát hiện hồi cứu. Một outlier lớn có thể ảnh hưởng tập tham chiếu. Muốn detector online cần tách cửa sổ fit và cửa sổ score, version hóa baseline và đánh giá drift; chưa có cơ chế đó trong project.

12.5 Từ khóa trong Description và bằng chứng tên sản phẩm

Không dùng phép tìm chuỗi con “STOCK” để gắn nhãn kho cho “STOCKING”. Code dùng full-word patterns và các biến thể được liệt kê. Tên tham chiếu của cùng StockCode phải xuất hiện trên ít nhất hai hóa đơn bán dương khác nhau, không thuộc phí/mã chữ đặc biệt; giữ mọi tên đạt điều kiện.

Nếu mô tả có từ khóa nhưng chính là tên tham chiếu, thêm keyword_product_name. Nếu có từ khóa nhưng không có đủ tham chiếu, thêm keyword_unresolved. Nếu có tham chiếu cùng mã và mô tả khác, có thể gắn context stock_words, damage_words, loss_words hoặc coding_words để hỗ trợ đối soát. Đây là suy luận ngữ cảnh; không tự sửa Description hay xóa dòng.

12.6 Trạng thái và bảo toàn dòng

Mỗi check có rule, status, reason, observed_value, reference_count, q1, q3 và upper_bound nếu áp dụng. Check status có flagged, not_flagged, not_applied. Trạng thái toàn dòng là flagged nếu có ít nhất một check flagged; nếu không có cờ và có ít nhất một check áp dụng không flagged thì not_flagged; nếu không có check áp dụng thì not_assessable.

Not_flagged ở cấp dòng không khẳng định mọi check đều đã áp dụng. Một dòng có thể vừa missing_customer_id vừa not_flagged ở cấp tổng vì một kiểm tra giá đã áp dụng bình thường. Các bảng kết quả phải giữ reason riêng của từng check để tránh diễn giải sai.

`validate_output` dùng count và exceptAll hai chiều trên tám cột gốc để bảo toàn multiset đầu vào trong cutoff. Nó kiểm tra đúng chín rule theo thứ tự và các status hợp lệ. Không chỉ so count: hai tập có cùng số dòng vẫn có thể mất một dòng và nhân đôi dòng khác.

12.7 Ngưỡng thống kê trên tổng hóa đơn

`assess_orders` chuẩn hóa invoice_key bằng trim/upper, loại những dòng không có khóa khỏi bảng tổng đơn và đếm riêng chúng. Một dòng hợp lệ cho tổng đơn phải có Quantity>0, UnitPrice hữu hạn >0, ngày hợp lệ trước cutoff và Country không blank. Tổng tiền dùng decimal rồi làm tròn hai chữ số.

Invoice được đánh giá nếu không hủy, không có dòng không hợp lệ, có tối đa một ID khách khác nhau (zero ID vẫn được phép), đúng một Country và đúng một ngày giao dịch. Một invoice có dòng sai sẽ có order_total=null, không dùng partial_total còn lại để chấm. Country hiện được đếm theo giá trị cột gốc; chỉ các kiểm tra blank/điều kiện hiện có được thực thi, chưa có bảng chuẩn hóa quốc gia.

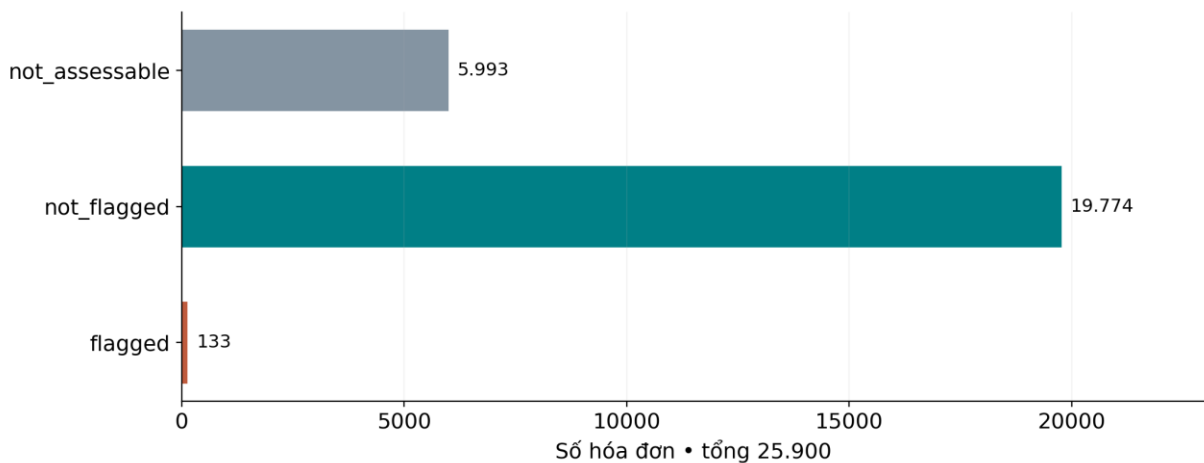
```
mean = trung bình order_total của các invoice đủ điều kiện
s = độ lệch chuẩn mẫu (stddev_samp)
upper_bound = mean + 3 * s
status = flagged khi order_total > upper_bound
```

Phải có ít nhất 30 invoice đủ điều kiện, mean/stddev hữu hạn và stddev>0. Không đủ điều kiện thì reason có thể là cancelled_invoice, invalid_invoice_lines, multiple_customer_ids, inconsistent_country, inconsistent_invoice_date, insufficient_reference hoặc zero_or_invalid_stddev. Giá trị bằng ngưỡng không bị flagged vì code dùng dấu lớn hơn nghiêm ngặt.

12.8 Ví dụ và kết quả thực tế

Một fixture có 40 đơn nhỏ quanh 10–12 GBP và một đơn BIG gồm hai dòng 500 GBP. Tổng BIG=1.000 GBP bị đánh dấu dù không có CustomerID. Một đơn BAD có một dòng dương lớn và một dòng giá âm nhận not_assessable và total=null; không bị đưa vào baseline như một đơn dương hợp lệ.

Run thật có 25.900 invoice: 133 flagged, 19.774 not_flagged, 5.993 not_assessable. Baseline đủ điều kiện gồm 19.907 invoice trong run này. Tỷ lệ flagged xấp xỉ 0,51% trên toàn invoice hoặc 0,67% trên invoice đủ điều kiện; luôn nêu rõ mẫu số. 40.051 là số dòng có ít nhất một row flag, không phải số hóa đơn gian lận.



Hình 6. Kết quả audit cấp hóa đơn; không đủ điều kiện được tách khỏi không có cờ.

13. Manifest, versioning và tính nhất quán khi chạy lại

13.1 Ba tầng metadata

Source manifest mô tả phiên bản nguồn. Run context đóng băng ngày, tham số và fingerprint nguồn/code của một DagRun. Stage manifest mô tả output đã ghi của ETL, RFM hoặc audit. Published manifest là con trỏ/snapshot metadata để consumer biết tập kết quả nào hoàn chỉnh cho cutoff.

Metadata	Trường quan trọng	AI ghi / AI đọc
source manifest	version, schema, coverage, file size/hash/rows	Bootstrap / Sensor và ETL
context.json	dag_id, run_id, run_key, D, C, source_hash, code_version, rule_version, k, churn	Validation / cả ba Spark stages và completion
etl.json	context_hash, outputs, metrics, complete	ETL / RFM, audit, completion
rfm.json, audit.json	context_hash, etl_hash, outputs, metrics	Nhánh analytics / completion
completion.json	context + ba stage manifests	Completion / kiểm chứng run
published/C.json	Metadata kết quả được công bố cho C	Completion / consumer

13.2 Định danh và fingerprint

run_key là SHA-256 của biểu diễn JSON dag_id/run_id. Nó tránh dùng nguyên run_id có ký tự không an toàn làm đường dẫn. code_version bám năm file business/shared code: retail_contracts, retail_pipeline và ba scripts legacy. Những thay đổi đang chưa commit cũng làm fingerprint thay đổi; fingerprint không thay thế commit SHA và không bao phủ toàn bộ OS/dependency/image.

Tên generation gồm phần đầu run_key và UUID mới. Hai attempts của cùng task có thể ghi hai generation khác nhau; chỉ generation có stage manifest thành công được tham chiếu. source_version giữ nhãn để đọc, source_hash giữ identity của toàn manifest. Chạy lại cùng ngày nhưng nguồn/code thay đổi là một run mới có provenance khác.

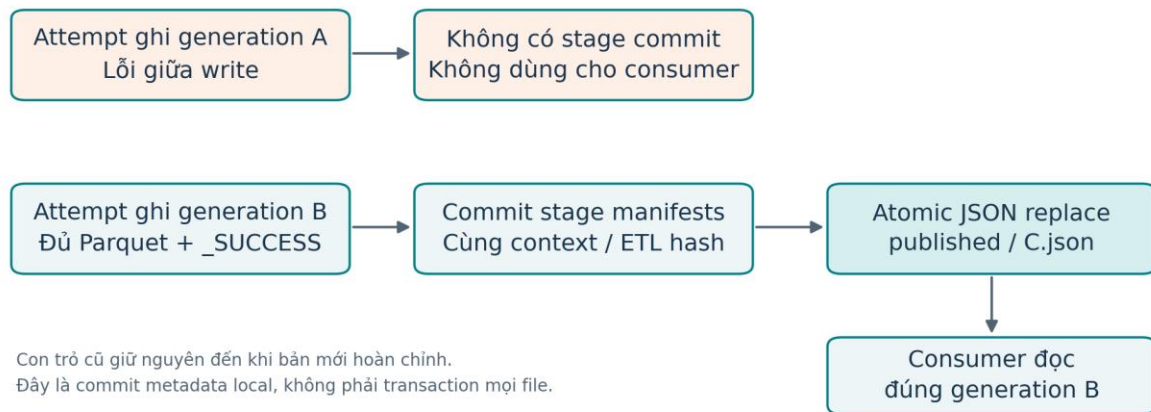
13.3 Idempotency đúng nghĩa trong project

Idempotency ở đây là retry/rerun không làm nhân đôi dữ liệu consumer theo kiểu append mù và không công bố output dở. Nó không có nghĩa output byte-for-byte giống nhau hoặc không tạo thêm thư mục. UUID, Parquet part layout và một số chi tiết Spark có thể khác; kết quả nghiệp vụ phải nhất quán với cùng input contract.

Giả sử audit lỗi sau khi RFM đã thành công. Published pointer của cutoff cũ vẫn giữ phiên bản hoàn chỉnh trước đó hoặc chưa tồn tại. Khi chạy lại audit với cùng context/ETL, completion có thể công bố khi đủ cả hai nhánh. Nếu ETL được clear và tạo generation mới thì cả hai nhánh phải chạy lại; analytics còn bám ETL cũ bị completion từ chối [B3, B12].

13.4 Atomic publication và giới hạn

atomic_json ghi file tạm cùng thư mục, flush, fsync rồi os.replace. Trước publication, _SUCCESS của từng output được kiểm tra. publication_lock dùng O_CREAT|O_EXCL theo cutoff để tránh hai publisher cùng thay pointer; lock được bỏ trong finally khi tiến trình kết thúc bình thường.



Hình 7. Output dở không trở thành bản công bố; chỉ metadata hoàn chỉnh được chuyển sang consumer.

Một process crash hoặc máy tắt có thể để lại lock và các generation mồ côi. Không tự xóa chúng khi chưa kiểm tra có writer còn hoạt động hay không. Atomic replace của một file không tạo transaction phân tán trên tất cả Parquet. `_SUCCESS` là marker ghi xong của Spark, không phải checksum đầy đủ từng part hay xác nhận người khác chưa sửa file sau đó.

13.5 Đọc kết quả đúng cách

Consumer bắt đầu từ `data/manifests/published/2011-12-10.json`, kiểm tra complete, chọn output tương ứng trong stages rồi đọc root của dataset. Root chứa year/month cần được giữ nguyên để Spark nhận cột partition. Không mở một part file bất kỳ rồi coi đó là toàn bộ kết quả.

```

$result = Get-Content data/manifests/published/2011-12-10.json -Raw | ConvertFrom-Json
$result.context
$result.stages.etl.metrics
$result.stages.rfm.metrics
$result.stages.audit.metrics
$result.stages.rfm.outputs.customers
  
```

Các đường dẫn output trong manifest tương đối với data root. Trong container, ghép `/opt/airflow/data/` với đường dẫn đó; trên host, ghép thư mục `data/` trong repository. Ví dụ cùng một path logic được volume ánh xạ sang hai namespace, không phải hai bản dữ liệu độc lập.

14. Hướng dẫn cài đặt từ máy mới

14.1 Chuẩn bị và kiểm tra vị trí làm việc

Cài Docker Desktop, bật Linux containers và backend WSL2 phù hợp máy Windows. Chuẩn bị khoảng 8 GB RAM có thể dành cho workload project và khoảng 20 GB đĩa trống cho image/build/output; nhu cầu thực tế tăng theo số snapshot giữ lại. Đảm bảo cổng 8080 chưa bị ứng dụng khác chiếm. Cần mạng để lấy base image, Debian packages, Python packages và dataset.

Không cần cài Airflow, Spark hoặc Java trực tiếp lên Windows để chạy workflow Docker này. `.venv` hiển thị trong terminal của IDE không điều khiển Python bên trong container. Nếu tạo môi trường host cho notebook hoặc lint, đó là môi trường riêng.

```
# Nếu chưa có checkout, clone repository của nhóm trước.
git clone <URL_REPOSITORY_CUA_NHOM>
Set-Location 'D:\HQC KỶ VII\Xử lý dữ liệu lớn\Airflow-Apache-Retail-E-Commerce-Dataset'
Get-Item docker-compose.yaml
docker version
docker compose version
docker info
```

Thay đường dẫn theo nơi bạn clone. `docker info` cần có cả Client lẫn Server; chỉ có Client chưa chứng minh Docker engine hoạt động. Lỗi “no configuration file provided” thường xuất hiện khi chạy Compose ở thư mục cha chứ không phải thư mục chứa YAML.

14.2 Tạo `.env` đúng nơi và đúng mục đích

Nếu `.env` chưa tồn tại, sao chép mẫu sau khi kiểm tra mẫu là cấu hình local an toàn. Nếu đã có `.env` đang chạy đúng, giữ lại và chỉ chỉnh những biến cần thiết. Chỉnh `.env.example` một mình không làm Compose tự nhận giá trị mới; Compose mặc định đọc `.env` ở project root.

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
docker run --rm apache/airflow:3.3.1 python -c "from cryptography.fernet import Fernet;
print(Fernet.generate_key().decode())"
docker run --rm apache/airflow:3.3.1 python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Lệnh thứ nhất tạo Fernet key; lệnh thứ hai tạo một JWT secret ngẫu nhiên. Khi Docker phải pull image, các dòng Digest: sha256 và Status: Downloaded newer image là thông tin image, không phải khóa cần dán. Giá trị key là dòng chuỗi do Python in ra. Không dùng một chuỗi Fernet mẫu công khai làm secret thật.

Biến trong <code>.env</code>	Cách chọn	Vai trò
<code>AIRFLOW_UID</code>	Thường 50000 với cấu hình đang dùng	UID chạy các container Airflow
<code>AIRFLOW_FERNET_KEY</code>	Chuỗi tạo bởi <code>cryptography.Fernet</code>	Mã hóa các giá trị được Airflow quản lý hỗ trợ Fernet
<code>AIRFLOW_API_JWT_SECRET</code>	Chuỗi ngẫu nhiên riêng	Secret cho cơ chế JWT/API của deployment
<code>AIRFLOW_ADMIN_USERNAME</code>	Tên đăng nhập local do bạn chọn	Tạo tài khoản lần init
<code>AIRFLOW_ADMIN_PASSWORD</code>	Mật khẩu riêng do bạn đặt	Tạo tài khoản lần init

Fernet không mã hóa toàn bộ PostgreSQL hay các file CSV/Parquet trên ổ đĩa. Thay key tùy tiện khi đã có dữ liệu mã hóa có thể làm mất khả năng giải mã; quy trình rotate là công việc riêng. Các service chia sẻ JWT secret qua anchor Compose, nên phải cập nhật nhất quán. Không in toàn bộ `.env` hoặc cấu hình Compose đã resolve vào báo cáo/screenshot.

14.3 Build và khởi tạo dịch vụ

```
docker compose build
docker compose up airflow-init
docker compose up -d
docker compose ps
```

Lần đầu build gồm tải base image, cài Java và requirements nên lâu hơn lần sau. Init phải kết thúc exit 0; nó không cần duy trì Up như scheduler. Bốn service dài hạn cần tiến tới healthy. Nếu build lỗi dependency, đọc log bước lỗi cụ thể; không lặp `build --no-cache` như bước đầu vì nó tải/cài lại toàn bộ và tốn đĩa.

Mở <http://localhost:8080>, đăng nhập bằng tài khoản đã được init. Nếu metadata volume đã có tài khoản, chỉ sửa AIRFLOW_ADMIN_PASSWORD trong `.env` rồi init lại không được xem như quy trình reset mật khẩu chắc chắn. Dùng cơ chế quản trị người dùng đúng của FAB và kiểm tra `airflow users --help` khi cần.

14.4 Lấy dữ liệu

Repository không chứa toàn CSV. Nếu `data/raw/data.csv` đã có, kiểm tra file rồi dùng nó; không tải từ một nguồn đang được source manifest tham chiếu. Phương án theo README là Kaggle CLI có sẵn trong image, sử dụng credentials của chính người dùng và mount read-only.

```
Get-Item data/raw/data.csv -ErrorAction SilentlyContinue
# Chỉ tải khi chưa có nguồn mong muốn.
docker compose run --rm --no-deps -v "${HOME}/.kaggle:/home/airflow/.kaggle:ro" airflow-scheduler bash -c
"kaggle datasets download -d carrier1/ecommerce-data -p /opt/airflow/data/raw --unzip"
Get-Item data/raw/data.csv
Get-Content data/raw/data.csv -TotalCount 1
```

Nếu CLI yêu cầu xác thực theo phiên bản Kaggle đang cài, cấu hình theo tài khoản/API hiện hành của Kaggle. Có thể tải ZIP thủ công từ trang dataset, giải nén đúng file `data.csv` vào `data/raw/`. Không đổi encoding hoặc sửa mô tả bằng Excel rồi ghi đè file nguồn mà vẫn giữ cùng source version.

Header mong đợi là InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country theo đúng thứ tự. File đang kiểm chứng khoảng 45,6 MB; kích thước chỉ là kiểm tra sơ bộ, SHA-256 và manifest mới dùng xác định identity nguồn cụ thể.

14.5 Bootstrap, connections và pool

```
docker compose exec airflow-scheduler python scripts/prepare_daily_landing.py
docker compose exec airflow-scheduler airflow pools set retail_spark 1 "One local Spark JVM"
docker compose exec airflow-scheduler airflow dags list-import-errors
docker compose exec airflow-scheduler airflow dags list
```

Bootstrap in ra path manifest khi xong. Connections `spark_default` và `retail_landing` đã được cấp bằng environment trong Compose, không cần tạo thêm bản khác trong UI. Giá trị environment có thể ưu tiên hơn giá trị sửa ở UI/database; muốn thay master hoặc landing root phải sửa nguồn cấu hình và recreate dịch vụ liên quan.

Pool là metadata Airflow; checkout code mới không tự tạo pool trong database. Nếu task queued lâu vì pool không tồn tại/không còn slot, kiểm tra pool trước khi nghỉ ngơi Spark. Healthcheck xanh không chứng minh pool đã được cấu hình.

14.6 Kiểm tra môi trường trước lần chạy đầu

```
docker compose exec airflow-scheduler airflow version
docker compose exec airflow-scheduler spark-submit --version
```

```
docker compose exec airflow-scheduler java -version
docker compose exec airflow-scheduler python -m pytest -q tests/test_retail_contracts.py
tests/test_retail_dag.py tests/test_retail_pipeline.py tests/test_pyspark_clean.py
tests/test_pyspark_rfm.py tests/test_pyspark_anomalies.py
```

Lệnh test đã cho kết quả 87 passed trong môi trường kiểm chứng. Tùy cấu hình máy, runtime có thể khác. `test_pyspark_jobs.py` kiểm tra toàn cấu trúc checkout nên chạy trên host có pytest hoặc gọi hàm test trực tiếp; runtime container không mount toàn bộ AGENTS/README/.agent nên không dùng test cấu trúc này để kết luận image thiếu file nghiệp vụ.

14.7 Chạy sau khi đã cài rồi

Đối với máy hiện tại, pipeline đã build, source đã bootstrap và pool đã tạo. Không cần làm lại các bước tạo key, ghi đè `.env`, download hoặc build nếu code/dependency không đổi. Mở Docker Desktop, vào đúng project, chạy `docker compose up -d`, kiểm tra `ps`, rồi thực hiện chương 15. Nhánh source code bind mount thay đổi sẽ được DAG processor đọc lại; thay dependency vẫn cần build.

15. Chạy, theo dõi và sử dụng kết quả

15.1 Trigger qua giao diện

Mở DAG `ecommerce_etl_dag`, bật trạng thái `unpaused` và chọn Trigger DAG. Nhập `business_date=2011-12-09`, `source_version=online-retail-v1`, `rfm_k=2`, `churn_days=90`. Chọn một `run_id` mới nếu giao diện cho nhập để phân biệt với run kiểm chứng. Không đổi cutoff sang ngày hiện tại chỉ vì đang demo hôm nay.

Theo dõi Graph để hiểu quan hệ, Grid để xem state/attempt theo run, và task logs để tìm lỗi hoặc summary. Một manual DagRun vẫn có thể queued khi DAG paused; CLI help của image cũng nêu điều đó. Bật DAG trước khi chờ task chạy. Sau demo có thể pause lại để giữ lịch được kiểm soát.

15.2 Trigger bằng PowerShell không lỗi dấu ngoặc JSON

Lệnh dưới dùng Python trong container tạo JSON argument, tránh các khác biệt quoting PowerShell với native executable. Đổi hậu tố `run_id` cho mỗi lần thử; không dùng lại một `run_id` đã tồn tại.

```
docker compose exec airflow-scheduler airflow dags unpause ecommerce_etl_dag
docker compose exec airflow-scheduler python -c "import json, subprocess;
subprocess.run(['airflow', 'dags', 'trigger', 'ecommerce_etl_dag', '--run-id', 'manual_demo_01', '--
conf', json.dumps(dict(business_date='2011-12-09', source_version='online-retail-
v1', rfm_k=2, churn_days=90))], check=True)"
docker compose exec airflow-scheduler airflow tasks states-for-dag-run ecommerce_etl_dag manual_demo_01 -o
json
docker compose exec airflow-scheduler airflow dags list-runs ecommerce_etl_dag -o json
```

Airflow 3.3.1 dùng `dag_id` dạng positional ở `dags list-runs`; ví dụ cũ dùng `-d` có thể bị báo unrecognized arguments. Khi chưa chắc flag, dùng `--help` trong đúng image. `airflow dags list` có DAG không có nghĩa run ETL đã thành công.

15.3 Cách đọc log theo tầng

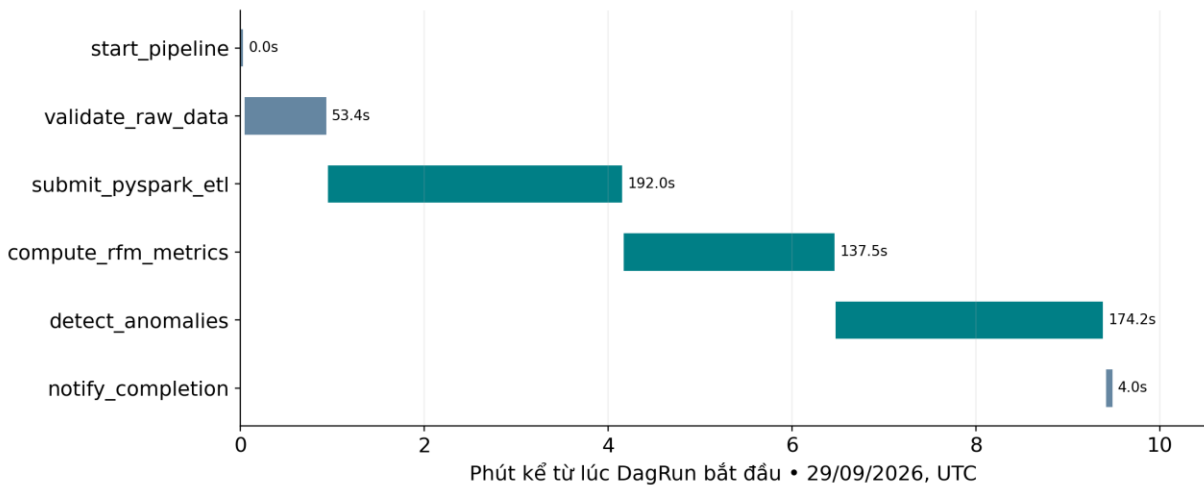
Cần tìm	Nơi xem	Câu hỏi trả lời
Docker engine/dịch vụ	<code>docker info</code> , <code>compose ps</code> , <code>compose logs</code>	Runtime và service có đang hoạt động không?
Lỗi parse DAG	<code>list-import-errors</code> , DAG processor log	Python import hoặc cấu hình có lỗi không?
Scheduler state/pool	Grid, CLI states, scheduler log	Task có đủ phụ thuộc/slot để chạy không?
Spark compute	Task log <code>SparkSubmitOperator</code>	Tiến trình Spark có exception, memory error hay hoàn tất không?
Kết quả mỗi stage	<code>STAGE_COMPLETE</code> và stage JSON	Đã ghi gì, bao nhiêu dòng, dùng context nào?
Công bố hoàn chỉnh	<code>PIPELINE_COMPLETE</code> , published JSON	Cả hai nhánh có cùng ETL generation không?

Task log trên host nằm dưới

`logs/dag_id=ecommerce_etl_dag/run_id=<run_id>/task_id=<task_id>/attempt=<n>.log`. `EmptyOperator` có thể được tối ưu hoàn tất mà không có một log compute như Spark task; kiểm tra state từ Airflow. Log JSON có timestamp, `dag_id`, `run_id`, `task_id` và `try_number` để truy vết.

15.4 Bảng chứng lần chạy thật

Run official_smoke_20260929 bắt đầu 06:17:58.986 UTC và kết thúc 06:27:28.054 UTC ngày 29/09/2026, khoảng 9 phút 29 giây. Cả sáu TaskInstance success. Không có retry trong bảng chứng run này; không được vẽ một retry giả để làm đẹp bài nộp.



Hình 8. Thời gian TaskInstance của run thực tế; hai nhánh Spark không overlap vì pool một slot.

Thời lượng theo stage manifest: ETL 181,010 giây, RFM 125,751 giây, audit 163,917 giây. Thời lượng TaskInstance lớn hơn vì có startup, submit và kết thúc process. Validation mất khoảng 53 giây trong run này do kiểm tra nhiều file trên bind mount; nó vẫn là kiểm tra metadata, không phải xử lý DataFrame.

15.5 Đọc Parquet bằng Spark

Không dùng Excel mở một part binary như CSV. Ví dụ sau đọc published manifest rồi hiển thị một số cột RFM; nó khởi tạo Spark session đọc kết quả, không chạy lại DAG.

```
import json
from pathlib import Path
from pyspark.sql import SparkSession

root = Path('/opt/airflow/data')
published = json.loads(
    (root / 'manifests/published/2011-12-10.json').read_text()
)
relative = published['stages']['rfm']['outputs']['customers']
spark = SparkSession.builder.master('local[2]').appName('ReadRFM').getOrCreate()
try:
    customers = spark.read.parquet(str(root / relative))
    customers.select(
        'CustomerID', 'Recency', 'Frequency', 'Monetary',
        'RFM_score', 'Cluster', 'segment', 'churn_assessment_status'
    ).show(20, truncate=False)
finally:
    spark.stop()
```

Đặt đoạn này trong một script đọc kết quả riêng nếu cần dùng nhiều lần; các đường dẫn trong ví dụ là đường dẫn container. Không lấy dataset vào driver bằng collect/toPandas toàn bộ nếu kích thước vượt RAM. Chỉ show/select một mẫu đủ cho báo cáo.

15.6 Chạy script legacy và khác biệt output

CLI legacy là cách chạy độc lập phục vụ kiểm tra/nghiên cứu; nó không tự tạo published manifest của pipeline chính thức. Cleaning legacy ghi đề hai thư mục mặc định; RFM và anomaly legacy ghi output

partition theo ngày trong root chỉ định. Phải kiểm tra đường dẫn output trước khi chạy để tránh thay thế kết quả người khác đang dùng.

```
docker compose exec airflow-scheduler spark-submit --master local[2] --driver-memory 3g
scripts/pyspark_rfm.py --input /opt/airflow/data/curated/RFM.parquet --output
/opt/airflow/data/analytics/rfm_legacy_demo --run-date 2011-12-10 --k 2
docker compose exec airflow-scheduler spark-submit --master local[2] --driver-memory 3g
scripts/pyspark_anomalies.py --input /opt/airflow/data/audit/anomalies.parquet --output
/opt/airflow/data/audit/anomaly_results_demo --run-date 2011-12-10
```

Chỉ dùng ví dụ khi các input legacy tồn tại và đúng version mong muốn. Các lệnh này không phải bước bắt buộc sau khi DAG chính thức đã chạy. Chạy cùng lúc job thủ công và DAG có thể vượt RAM vì job thủ công không bị pool Airflow chặn.

15.7 Dừng và chạy lại dịch vụ

```
docker compose exec airflow-scheduler airflow dags pause ecommerce_etl_dag
docker compose stop
# Lần sau:
docker compose up -d
```

Pause ngăn lịch/task mới được tiếp tục xét theo cơ chế Airflow; không xem nó là lệnh bảo đảm dừng ngay một Spark process đang chạy. Nếu muốn dừng môi trường, chờ run hoàn tất hoặc xử lý hủy có chủ đích. Không kill JVM/ngắt Docker giữa một job chỉ để “refresh” giao diện.

16. Kiểm thử, đối soát và bằng chứng nghiệm thu

16.1 Các lớp kiểm thử

Lớp	Kiểm tra gì?	Bằng chứng hiện có
Static / lint	Cú pháp, import đơn giản, style, topology	Ruff, formatter, AST và diff check
Unit contract	Date, hash, path, manifest, publication	Fixture không cần toàn dataset
Unit DAG/Sensor	Operator thật, phụ thuộc, reschedule/fail-fast, timetable	Test dùng API image thực và mock điểm giao tiếp
Spark logic	Cleaning, RFM, scores, IQR, order/churn boundary	Các module pytest Spark
Scheduler integration	Airflow thực thi cả sáu task	official_smoke_20260929 success
Read-back	Parquet và metadata sau ghi	Tám output khớp count, 13 monthly partitions

Một test pass không thay thế mọi lớp còn lại. AST parse không chứng minh provider có class đúng; import DAG không chứng minh source tồn tại; Spark chạy tay không chứng minh scheduler/XCom hoạt động; task success không tự chứng minh tất cả số liệu đầu ra đúng. Đây là lý do project dùng nhiều lớp kiểm chứng [B10].

16.2 Bộ test và giới hạn fixture

test_pyspark_anomalies.py

Có 7 hàm test khai báo; parametrize có thể làm số test cases thực thi lớn hơn.

- cli_rejects_bad_parameters_and_overlapping_paths
- contract_cutoff_quality_and_business_checks
- whitespace_only_identifiers_and_text_are_missing
- description_context_uses_two_distinct_invoices_and_same_stock
- exact_iqr_boundaries_and_exclusions
- exact_interpolation_minimum_30_and_duplicate_rows
- empty_and_partition_round_trip

test_pyspark_clean.py

Có 5 hàm test khai báo; parametrize có thể làm số test cases thực thi lớn hơn.

- default_output_paths_follow_project_layout
- read_transactions_uses_stable_business_types
- prepare_datasets_applies_each_output_rule
- prepare_datasets_excludes_service_lines_only_from_rfm
- run_cleaning_job_writes_both_parquet_outputs

test_pyspark_jobs.py

Có 1 hàm test khai báo; parametrize có thể làm số test cases thực thi lớn hơn.

- required_project_structure_exists

test_pyspark_rfm.py

Có 37 hàm test khai báo; parametrize có thể làm số test cases thực thi lớn hơn.

- validate_curated_contract_accepts_canonical_data
- validate_curated_contract_rejects_noncanonical_schema
- validate_curated_contract_rejects_missing_required_columns
- validate_curated_contract_rejects_unclean_values
- validate_curated_contract_rejects_empty_input
- validate_curated_contract_rejects_null_values
- validate_curated_contract_rejects_blank_ids
- validate_curated_contract_rejects_negative_or_zero_values
- direction_and_boundaries
- ties_and_id_independence
- all_equal_uses_rank_zero
- only_constant_metric_gets_one
- add_rfm_segment_requires_explicit_k
- add_rfm_segment_rejects_k_below_two
- add_rfm_segment_rejects_k_larger_than_customer_count
- add_rfm_segment_rejects_insufficient_distinct_features
- add_rfm_segment_rejects_fewer_occupied_clusters
- add_rfm_segment_groups_similar_customers_together
- select_history_rejects_no_transactions_before_cutoff
- select_history_boundary_exact_cutoff
- select_history_prunes_partitions_when_year_month_present
- select_history_without_partition_columns_still_filters_by_date
- compute_rfm_separates_multiple_customers
- compute_rfm_rounds_monetary_half_up
- compute_rfm_rounds_after_summing_line_totals
- metrics_and_cutoff
- validate_rfm_output_rejects_missing_cluster
- validate_rfm_output_rejects_invalid_cluster
- validate_rfm_output_rejects_noninteger_cluster_type
- validate_rfm_output_accepts_cluster_boundaries
- output_validation_requires_valid_k
- write_rfm_rejects_invalid_cluster_before_writing
- write_rfm_writes_partitioned_parquet_and_overwrites
- write_rfm_overwrite_preserves_other_dates
- nonfinite_price_rejected
- cli_date
- cli_k_is_required_and_validated

test_retail_contracts.py

Có 8 hàm test khai báo; parametrize có thể làm số test cases thực thi lớn hơn.

- bootstrap_roundtrip_empty_day_and_idempotency
- committed_source_cannot_change
- missing_empty_corrupt_files_and_invalid_day

- `paths_and_date_reject_unsafe_inputs`
- `checksum_detects_same_size_change`
- `context_is_frozen_and_cutoff_is_next_day`
- `atomic_replace_failure_preserves_published_file`
- `publish_rejects_mixed_generations_and_missing_markers`

test_retail_dag.py

Có 5 hàm test khai báo; `parametrize` có thể làm số test cases thực thi lớn hơn.

- `topology_real_operators_and_success_only_leaf`
- `manual_requires_date_and_scheduled_forbids_wrong_override`
- `timetable_last_interval_and_end_boundary`
- `sensor_reschedule_never_creates_context`
- `sensor_ready_validates_and_invalid_source_fails_without_retry`

test_retail_pipeline.py

Có 4 hàm test khai báo; `parametrize` có thể làm số test cases thực thi lớn hơn.

- `csv_quotes_unicode_and_parse_quarantine`
- `order_total_missing_customer_and_invalid_line`
- `sparse_or_constant_reference_is_not_a_negative_finding`
- `churn_threshold_priority_and_insufficient_history`

Fixture `sample_transactions.csv` có ví dụ ngày năm 2026 phục vụ unit logic, không phải nguồn lịch sử dùng trigger official DAG. Không đặt fixture đó dưới `source_version` `online-retail-v1` để chạy lịch bounded 2010–2011. Test lỗi có chủ đích không phải dữ liệu thật bị hỏng.

16.3 Kết quả kiểm chứng đã có

87 tests passed trong 281,51 giây; một `FutureWarning` upstream về hỗ trợ `pandas 3` của `PySpark`, không phải test failed. Ruff check `dags scripts tests` đã qua. Cấu hình lint có ngoại lệ hẹp E501/E402 cho một số file legacy; không tuyên bố mọi dòng legacy đã được formatter viết lại.

Đọc lại Parquet bằng Spark session độc lập xác nhận `transactions=391.057`, `audit_input=536.641`, `parse_errors=0`, `customers=4.334`, `cluster_profile=2`, `row_assessments=536.641`, `order_assessments=25.900`, `flagged_orders=133`. CSV nguồn có SHA-256 khớp source manifest. Các con số không phải count của file CSV header hay count part files.

16.4 Những kiểm thử chưa được làm

Chưa cố tình chạy một `DagRun` quá 60 phút để quan sát deadline callback; chưa chạy nhiều ngày backfill qua scheduler; chưa kill JVM giữa write rồi chạy retry recovery; chưa benchmark hai nhánh overlap; chưa benchmark dữ liệu nhân rộng hàng triệu dòng/ngày. Đây là các thí nghiệm tiếp theo có thiết kế trong tài liệu DAG, không được ghi thành “passed” trong báo cáo này.

Nếu làm thí nghiệm crash, dùng `source version/namespace` riêng, giữ một published pointer cũ đã biết tốt, gây lỗi ở một stage, xác nhận pointer không đổi, rồi clear đúng downstream và đọc lại. Thí nghiệm phải ghi thời điểm, revision, tác động và kết quả; không thử trên một output đang được consumer thật sử dụng.

16.5 Bộ bằng chứng để nộp

Giữ revision/source/code fingerprint, versions, lệnh test, output pytest/lint, trạng thái sáu task, summary JSON, schema và screenshot Graph/Grid nếu cần. Khi chụp màn hình, chọn đúng run_id/cutoff; không dùng ảnh DAG placeholder từ trước. Log summary có thể đáp ứng phần evidence logs của rubric; Gantt trong tài liệu là hình dựng từ timestamp thật, được ghi nhãn rõ.

17. Xử lý sự cố và phục hồi

17.1 Quy trình chẩn đoán theo thứ tự

Xác định lỗi nằm ở host, engine, container, DAG import, scheduling, validation, Spark hay publication. Đọc thông báo đầu tiên có nguyên nhân cụ thể và phần cuối stack trace; đừng chỉ chụp một dòng INFO bất kỳ. Ghi run_id/task_id/attempt trước khi clear để không mất mối liên hệ giữa nguyên nhân và bằng chứng.

Triệu chứng	Cách kiểm tra	Hướng xử lý phù hợp
no configuration file provided	Get-Location; Get-Item docker-compose.yaml	Vào đúng project root hoặc dùng -f với đường dẫn YAML
docker info chỉ hiện Client	Docker Desktop trạng thái, Server, ổ đĩa	Chờ/khởi động engine, kiểm tra WSL và dung lượng; chưa cần sửa DAG
Build đứng ở exporting to image	Docker Desktop, docker info, disk/CPU	Export/unpack có thể lâu; kiểm tra tiến triển, không build chồng nhiều lần
Ổ C đầy	Free space; nơi lưu Docker disk image	Dọn dữ liệu đã xác định hoặc dùng Settings → Disk image location sang D nếu có; không tự di chuyển/xóa VHDX
Container unhealthy	compose logs đúng service, healthcheck	Sửa migration/connection/resource cụ thể; restart chỉ sau khi hiểu lỗi
DAG không thấy / import error	list-import-errors; PYTHONPATH; provider versions	Sửa import/path; recreate khi env đổi; build khi dependency đổi
Manual run queued mãi	Paused, pool, active runs, scheduler	Unpause, tạo retail_spark, chờ run trước hoặc sửa scheduler
Sensor reschedule	Manifest path, source_version, producer	Công bố nguồn đúng; chờ hữu hạn, không bấm mark success
Outside source coverage	D và first/last_date	Dùng ngày lịch sử hợp lệ hoặc version nguồn mới
Size/header/checksum mismatch	File bị sửa/thiếu, manifest version	Điều tra nguồn; tạo version mới nếu nguồn thay đổi có chủ đích
Business code changed / inputs changed	Context và fingerprint	Tạo DagRun mới; không sửa JSON context để vượt kiểm tra
Spark OOM / exit 137	Task logs, docker stats, số JVM	Chờ job tay, giảm concurrency, cấp đủ RAM; không tăng parallelism khi thiếu bộ nhớ
KMeans không đủ khách/vector	Cutoff, k, dữ liệu clean	Chọn k phù hợp hoặc phạm vi dữ liệu đủ; không tạo cụm giả
Different ETL generations	etl_hash hai nhánh	Chạy lại cả hai nhánh sau ETL generation mới
Publication lock tồn tại	Có run/process đang ghi không?	Chỉ gỡ lock khi xác minh là stale; không xóa tự động

17.2 Các lệnh thu thập thông tin không thay dữ liệu

```
docker compose ps
docker compose logs --tail 100 airflow-scheduler
docker compose logs --tail 100 airflow-dag-processor
docker compose exec airflow-scheduler airflow dags list-import-errors
docker compose exec airflow-scheduler airflow tasks states-for-dag-run ecommerce_etl_dag <RUN_ID> -o json
docker stats --no-stream
Get-PSDrive -PSProvider FileSystem
```

Không chia sẻ toàn bộ log/environment nếu có token hoặc credential. Gửi phần lỗi có ngữ cảnh cần thiết. Warning “No Partition Defined for Window operation” trong RFM phản ánh Window xếp hạng toàn quần

thể khách, là giới hạn hiệu năng đã biết; nó không tự có nghĩa job thất bại. NativeCodeLoader warning không có thư viện native Hadoop cũng không tự chứng minh lỗi dữ liệu khi Spark dùng Java fallback.

17.3 Chọn retry, clear hay run mới

Tình huống	Hành động	Vì sao
Lỗi tạm thời, input/code không đổi	Để retry hoặc clear task failed và downstream cần thiết	Context vẫn hợp lệ
RFM lỗi nhưng ETL không đổi	Chạy lại RFM; completion chạy khi đủ cả hai nhánh	Audit đã bám đúng ETL vẫn có thể dùng
ETL được chạy lại tạo generation mới	Chạy lại cả RFM và audit, rồi completion	Hai nhánh phải đọc cùng generation
Sửa business code hoặc ngưỡng Params	Trigger run mới	code/context fingerprint của run cũ cố định
Sửa CSV nguồn	Bootstrap source version mới và run mới	Không ghi đè version được gọi là immutable
Consumer đang đọc kết quả trước đó	Giữ pointer cũ cho đến bản mới hoàn chỉnh	Không để consumer đọc partial write

17.4 Sao lưu và dọn dữ liệu

Backup cần gồm nguồn/manifest cần tái lập, output published cần giữ và metadata database nếu muốn giữ lịch sử UI/permissions. Copy thư mục đang được Spark ghi không bảo đảm bản backup nhất quán. Snapshot output versioned chỉ là cách tổ chức kết quả, không thay thế backup ở nơi lưu độc lập.

Chưa có retention/garbage collection tự động. Một generation không được pointer hiện tại trỏ đến vẫn có thể được completion lịch sử tham chiếu. Trước khi dọn cần xây dựng danh sách tham chiếu và thời hạn giữ; không dùng `docker system prune --volumes` hay xóa toàn data để sửa một task lỗi. Chương này mô tả cách cân nhắc, không yêu cầu thực hiện xóa.

18. Bảo mật, tái lập và giới hạn mở rộng

18.1 Các biên bảo mật thực tế

`.env` bị ignore nhưng vẫn là plaintext trên host. Credential trong environment container không được Fernet tự bảo vệ toàn diện. Compose hiện là cấu hình local phục vụ học tập, có tài khoản PostgreSQL local trong YAML và publish API port lên host; chưa có reverse proxy TLS, mạng zero-trust, secret manager hay hardening production.

Bind mounts chia sẻ file giữa các process có quyền ghi. `safe_path` ngăn đường dẫn người dùng thoát root theo resolve và token kiểm soát `source_version`; đó không phải sandbox hoàn chỉnh chống một process có quyền ghi root. Trusted deployment configuration phải do người quản trị kiểm soát.

Khi triển khai ngoài máy học tập, cần phân quyền, quản lý secret tập trung, TLS, giới hạn network, backup/restore, audit access, image scanning và quy trình cập nhật dependency. Các nguyên tắc này dựa trên nhu cầu vận hành và sách [B12, B16]; không coi chúng đã được triển khai chỉ vì có tên trong báo cáo.

18.2 Reproducibility có những tầng nào?

Để tái lập kết quả cần cùng dữ liệu nguồn, rule/code, cutoff/params, dependency/runtime và điều kiện tính toán liên quan. Source SHA, manifest hash, code fingerprint và k/churn đã được lưu. Image digest/complete dependency lock, model artifacts và full hardware manifest chưa được lưu thành một gói tái lập hoàn chỉnh.

Seed cố định giúp KMeans ổn định hơn nhưng không hứa byte-identical giữa các phiên bản Spark, partitioning hoặc phép tính số khác nhau. Cluster IDs tùy ý cũng cần được căn chỉnh theo nội dung cụm khi so sánh các ngày, thay vì so đơn giản ID 0 với ID 0.

18.3 Các điểm nghẽn khi tăng dữ liệu

Điểm nghẽn	Vì sao xuất hiện	Hướng cải tiến cần đánh giá
Bootstrap giữ toàn nguồn trong RAM	csv rows được nhóm thành list	Ingestion phân tán/streaming theo chunk với commit contract
Full-history snapshot mỗi D	Lập đọc/ghi lịch sử cho mọi cutoff	Fact table và incremental aggregation/recompute có phạm vi
Window rank toàn khách	Global ordering về một partition	Quantile/bucket phân tán nếu chấp nhận đổi semantics
Exact percentile theo sản phẩm	Nhóm rất lớn tốn compute/memory	Approximate quantile có sai số được đo và công bố
Nhiều file nhỏ	Daily source, monthly output x nhiều versions	Compaction/partition sizing theo workload
Bind mount Windows	Metadata I/O chậm khi nhiều file	Storage Linux/remote phù hợp; đo thay vì đoán
Compute cùng scheduler	CPU/RAM cạnh tranh	Tách worker/compute cluster và monitoring riêng
Local lock/replace	Không chuyển nguyên xi sang object storage	Conditional write/transactional table format thích hợp

18.4 SLO và observability tiếp theo

Theo dõi duration theo stage, queue delay, hàng chờ pool, số dòng nguồn/dedup/clean, tỷ lệ parse error, tỷ lệ invoice not_assessable và độ mới của published cutoff. Mức tăng đột biến của not_assessable có thể

do nguồn hổng, không phải phát hiện “ít fraud hơn”. Hiện code log/counts đã tạo dữ liệu cho các chỉ số nhưng chưa có dashboard Prometheus/Grafana hay cảnh báo ngoài hệ thống.

Có thể bổ sung freshness watchdog, deadline smoke test, service failure drill và replay/backfill test nhỏ. Mỗi cải tiến cần tiêu chí đạt, kế hoạch rollback và dữ liệu riêng phù hợp. Không tự bổ sung Kafka hay cluster chỉ để nhiều công nghệ hơn khi bài toán và môi trường chưa cần.

19. Tổng hợp kết quả và ma trận đáp ứng yêu cầu

19.1 Kết quả từ artifact thực tế

Chỉ số / định danh	Giá trị
DagRun	official_smoke_20260929
Business date / cutoff	2011-12-09 / 2011-12-10
Source version	online-retail-v1
Task success	6/6
Tests	87 passed; 1 upstream warning
Raw / dedup / clean	541.909 / 536.641 / 391.057
Parse errors	0
Khách RFM / cụm	4.334 / 2
High-value / inactive	1.734 / 1.463
Hóa đơn: flagged / not_flagged / not_assessable	133 / 19.774 / 5.993
Dòng có cờ / không có cờ	40.051 / 496.590
Monthly partitions	13
Independent read-back	passed
Source SHA-256	d07aec9960083af2339975a3f9d3b26313b342dcd9f86cce0b919b1cde639a44
Code fingerprint	09fec8a1b9b9cdc19e0113e19560102ac7a9be7125d80b9b7d0c30ffc34a79a9

Số liệu lấy từ [docs/evidence/official_run_20260929.json](#), stage manifests và read-back, không nhập tay từ một screenshot không truy vết. Kết quả kiểm chứng của ngày/cutoff khác phải được lưu riêng. Bản report Word này không tự chạy lại pipeline khi mở file.

19.2 Ánh xạ yêu cầu chức năng của DOCX

Yêu cầu	Hiện thực / vị trí giải thích	Mức bằng chứng
Giải thích cơ chế Airflow	Chương 4 và 7–8	Phân tích khái niệm gắn code thực
Phân biệt orchestration và compute	Chương 2–3, sơ đồ kiến trúc	Đúng topology Docker/Spark local
Dataset e-commerce tám trường	Chương 9, schema nguồn	Nguồn 541.909 dòng và SHA
Đúng sáu task và fan-out/fan-in	Chương 8, full DAG phụ lục	Topology tests và scheduler run
Kiểm tra partition/size trước compute	Chương 9, Sensor	Test file/date/hash; validation thật success
PySpark cleaning và chuẩn hóa kiểu	Chương 10	Fixture/regression và Parquet read-back
Parquet year/month	Chương 10, dictionary phụ lục	13 monthly partitions đã đọc lại
RFM bằng aggregation/Window	Chương 11	Test thuật toán và 4.334 output khách
Segmentation và inactive accounts	Chương 11	KMeans + rule labels; proxy, chưa có mô hình churn

Yêu cầu	Hiện thực / vị trí giải thích	Mức bằng chứng
Order totals vượt ngưỡng thống kê	Chương 12	133 invoice flagged; có trạng thái/reason
Completion logs hoặc email	Chương 8 và 15	Log PIPELINE_COMPLETE; không gửi email
Setup có thể làm theo	Chương 14–17, Compose phụ lục	Môi trường hiện tại đã chạy; checkout mới chưa smoke lại toàn bộ trong lượt viết Word
Code đúng cấu trúc và tests	Chương 6, 16 và phụ lục hàm	File thực tế, 87 tests, lint
Trình bày/live demo	Chương 20	Có kịch bản và bằng chứng; slide riêng chưa tạo

19.3 Đáp ứng rubric mới mà không tự chấm điểm

Nhóm lý thuyết có giải thích gắn ví dụ và nguồn sách. Nhóm DAG có real operators, Sensor, date/retry/deadline và dependency rõ. Nhóm Spark có schema, DataFrame/Window, outputs partitioned và kiểm thử. Nhóm tái lập có Compose, cấu trúc, hướng dẫn và logs. Nhóm chất lượng có quarantine, null policy, row preservation và order audit.

Điểm cần nói thẳng khi đánh giá là hai nhánh chưa overlap ở runtime, deadline quá hạn chưa được test end-to-end, chưa thực hiện backfill nhiều ngày và chưa scale benchmark. Đề nhắc workflow song song; topology đã có fan-out/fan-in, nhưng nếu giảng viên yêu cầu bằng chứng thực thi song song thật thì còn cần thí nghiệm pool=2 với tài nguyên đủ. Không đánh dấu đạt 100 điểm chỉ vì tài liệu bao phủ tên tiêu chí.

19.4 Phần còn lại trước bài nộp cuối

Chọn ảnh Graph/Grid có run_id, chọn log summary vừa đủ đọc, hoàn thiện slide và phân công thuyết trình, chạy một buổi demo thử theo tài nguyên máy lớp. Nếu yêu cầu PDF/report path cụ thể, có thể nộp bản PDF đi kèm Word cùng REPORT.md điều hướng. Việc push repository là thao tác riêng; chưa được thực hiện trong công việc tạo tài liệu này.

20. Kịch bản demo và câu hỏi bảo vệ

20.1 Kịch bản khoảng 15 phút

Thời gian	Nội dung	Bảng chứng / thông điệp
0–2 phút	Bài toán và nguồn dữ liệu	Dòng hóa đơn khác đơn/khách; cần tự động hóa
2–5 phút	Sơ đồ hai plane và Docker	Airflow điều phối, Spark compute; local không phải cluster
5–7 phút	DAG, ngày và Sensor	Sáu task; D/C; manifest ready; pool
7–10 phút	RFM và anomaly	Công thức, distinct invoice, missing ID, ngưỡng total GBP
10–12 phút	Mở run success và output	Grid/log/manifest; số liệu thật
12–14 phút	Retry/idempotency/tests	Không publish partial; 87 test và read-back
14–15 phút	Giới hạn và mở rộng	Parallel run/deadline/scale chưa kiểm chứng

Không cần đợi toàn job 9–10 phút mới bắt đầu giải thích: có thể dùng run đã hoàn tất để chứng minh, rồi trigger một run mới nếu thời gian và tài nguyên cho phép. Nếu dùng run đã có phải nói rõ đó là run lưu trước, không mô tả thành đang xử lý trực tiếp.

20.2 Các câu hỏi nên trả lời được

Câu hỏi	Trả lời cốt lõi
Vì sao không truyền DataFrame bằng XCom?	Metadata nhỏ phù hợp orchestration; dữ liệu lớn đi qua storage với schema/manifest
Vì sao DAG daily nhưng đọc toàn lịch sử?	RFM là as-of cumulative; snapshot độc lập đổi lấy chi phí đọc/ghi lặp
Vì sao Recency thấp nhất là 1?	Cutoff đầu ngày sau D và điều kiện InvoiceDate < C
Vì sao Frequency không count rows?	Một invoice có nhiều sản phẩm; đề cần số đơn phân biệt
Thiếu ID có phải bỏ mọi phân tích?	Bỏ khỏi customer aggregation, giữ cho audit có đủ trường
Idempotent mà có nhiều version thư mục?	Consumer chỉ đọc bản commit; không append đôi; không hứa filesystem không đổi
Vì sao hai task độc lập không chạy cùng lúc?	Pool một slot theo ngân sách RAM; dependency và resource scheduling khác nhau
Vì sao không dùng SLA cũ?	Version 3.3.1 có DeadlineAlert; hard timeout và freshness là các khái niệm riêng
Vì sao không áp \$5.000?	Nguồn GBP; đề nêu ví dụ, project chọn mean+3s đúng đơn vị nguồn
133 đơn có phải fraud?	Chỉ vượt ngưỡng hồi cứu, cần rà soát và nhãn xác thực
Cluster 0 có phải VIP?	ID tùy ý; nhãn business tính độc lập bằng R/F/M scores
Container healthy có đủ chứng minh bài chạy?	Không; cần task success, metrics và output read-back

20.3 Bài thực hành đọc code

Bắt đầu từ DAG và tìm spark_job, Jinja context path, Params và dependency. Sang sensor để xem ngày manual/scheduled và điểm tạo context. Sang retail_contracts để xem identity/commit. Sang retail_pipeline để xem stage dispatch/output. Cuối cùng đọc các hàm transform trong ba script legacy và tests tương ứng. Tuyến này đi theo luồng thực thi thay vì đọc mọi file theo thứ tự alphabet.

21. Nguồn tham khảo và căn cứ thiết kế

21.1 Sách người dùng cung cấp

Julian de Ruiter, Ismael Cabral, Kris Geusebroek, Daniel van der Ende và Bas Harenslak, Data Pipelines with Apache Airflow: Orchestration for Data and AI, Second Edition, Manning. Trang dưới là trang in; file PDF người dùng cung cấp có offset +28 trang. Tài liệu này diễn giải và áp dụng nguyên tắc, không sao chép nguyên chương.

Mã	Chương/mục; trang in	Trang PDF	Áp dụng
B1	Ch.1 §1.1–1.3; 4–19	32–47	DAG, phạm vi điều phối
B2	Ch.2 §2.2/2.5; 27–29, 38–40	55–57, 66–68	Operators/tasks/failures
B3	Ch.3 §3.4–3.7; 55–66	83–94	Interval, backfill, idempotency
B4	Ch.4 §4.2–4.8; 70–85	98–113	Asset scheduling, phương án khác
B5	Ch.5 §5.2–5.3; 89–101	117–129	Templating và runtime context
B6	Ch.6 §6.1/6.4–6.5; 112–114, 128–136	140–142, 156–164	Dependencies, trigger rules, XCom
B7	Ch.7 §7.1; 148–156	176–184	Sensors, timeout, reschedule
B8	Ch.8 §8.3.3; 185–186	213–214	Offload compute và SparkSubmit
B9	Ch.9 §9.2/9.4; 197–204, 210–212	225–232, 238–240	Connections/hooks/custom sensors
B10	Ch.10 §10.1/10.4; 227–240, 251–255	255–268, 279–283	Các lớp kiểm thử
B11	Ch.11 §11.2–11.5; 264–287	292–315	Containers/volumes/deployment
B12	Ch.12 §12.1–12.4; 296–320	324–348	Best practices, storage, pool
B15	Ch.15 §15.1–15.8; 383–418	411–446	Executor, concurrency, monitoring
B16	Ch.16 §16.1–16.5; 425–440	453–468	Security và credentials

Các con số 3g, local[2], timeout 20/30 phút, churn 90 ngày, k=2 và minimum samples=30 là cấu hình/quy tắc của project, không phải con số phổ quát được sách yêu cầu. Khi giải thích lựa chọn cần nêu dataset, tài nguyên và giới hạn tương ứng.

21.2 Tài liệu chính thức và nguồn local

Mã	Nguồn	Cách sử dụng
W1	https://airflow.apache.org/docs/apache-airflow/3.3.1/howto/deadline-alerts.html	DeadlineAlert, experimental status, SyncCallback
W2	https://airflow.apache.org/docs/apache-airflow/3.3.1/authoring-and-scheduling/timetable.html	Khái niệm timetable; API đã đối chiếu thêm trong image
W3	https://airflow.apache.org/docs/apache-airflow-providers-apache-spark/stable/operators.html	SparkSubmitOperator; durable execution cần phân biệt local/remote
W4	https://airflow.apache.org/docs/apache-airflow-providers-standard/stable/_api/airflow/providers/standard/sensors/filesystem/index.html	FileSensor và filesystem connection

Mã	Nguồn	Cách sử dụng
W5	https://airflow.apache.org/docs/apache-airflow/3.3.1/cli-and-env-variables-ref.html	CLI; flag thực tế lấy từ --help của image đang chạy
W6	https://archive.ics.uci.edu/dataset/352/online+retail	Dataset và đơn vị giá sterling
W7	https://www.kaggle.com/datasets/carrie1/ecommerce-data	Bản dataset được project chọn
L1	airflow_subject.docx và rubric cập nhật của người dùng	Yêu cầu chức năng và trọng số đánh giá
L2	DAG, scripts, Dockerfile, Compose và requirements trong repository	Nguồn sự thật về implementation
L3	docs/evidence/official_run_20260929.json	Kết quả tổng hợp, phiên bản, read-back
L4	docs/DAG_DESIGN_VI.md; docs/IMPLEMENTATION_VI.md	Quyết định, traceability và giới hạn đã ghi nhận
L5	Task logs của official_smoke_20260929	Thời gian thực thi, stage summaries và completion

Các URL “stable” có thể chuyển sang provider mới theo thời gian. Khi khác biệt với ví dụ trên web, kiểm tra version đang cài, signature và help CLI trước; báo cáo này mô tả Airflow 3.3.1 cùng provider versions đã ghi rõ. Những trang web chỉ dùng hỗ trợ; cấu hình và kết quả project được đọc trực tiếp từ file local.

Phụ lục A. Từ điển thuật ngữ và cấu hình nhanh

Thuật ngữ	Ý nghĩa trong bài
DAG	Đồ thị dependency không chu trình
DagRun	Một lần chạy DAG có run_id và state
TaskInstance	Task cụ thể trong một run/attempt
Operator	Lớp định nghĩa loại công việc
Sensor	Operator chờ điều kiện
Hook	Client tích hợp hệ thống
Connection	Cấu hình kết nối có ID
XCom	Giá trị nhỏ trao đổi giữa task instances
Params	Tham số DAG có schema và giá trị theo run
Executor	Cơ chế thực thi Airflow task
Pool	Giới hạn tài nguyên logic qua số slot
Data interval	Khoảng dữ liệu mà scheduled run đại diện
Logical date	Định danh thời gian logic; không luôn là thời điểm chạy thật
Backfill	Tạo run cho khoảng lịch sử
Catchup	Chính sách scheduler tạo interval bị bỏ lỡ
Idempotency	Rerun/retry không làm sai kết quả consumer do tác dụng phụ lặp
Atomic publication	Chuyển con trỏ metadata hoàn chỉnh như một thao tác thay thế
Manifest	Metadata về nguồn/output, version, paths và metrics
Generation	Một phiên bản output riêng của một lần thử
Partition pruning	Bỏ đọc partition không phù hợp điều kiện
Shuffle	Phân phối lại dữ liệu giữa partition để group/join/sort
Spark driver	Tiến trình quản lý Spark application
Spark task	Đơn vị công việc bên trong Spark, khác Airflow task
Window	Phép tính trên cửa sổ/sắp hạng các hàng liên quan
RFM	Recency, Frequency, Monetary
IQR	Khoảng tứ phân vị Q3-Q1
Inactivity proxy	Dấu hiệu dựa trên thời gian chưa mua, không phải xác suất churn
Not assessable	Không đủ điều kiện kết luận ở cấp tổng
Not applied	Một check không áp dụng với record cụ thể
Read-back	Đọc lại output thực tế để đối chiếu contract/số liệu
Bind mount	Ánh xạ đường dẫn host vào container
Named volume	Lưu trữ được Docker quản lý theo tên

Tên	Giá trị	Nơi/quy tắc
DAG ID	ecommerce_etl_dag	dags/ecommerce_etl_dag.py
Schedule / timezone	0 0 * * * / UTC	CronDataIntervalTimetable
start_date / end_date	2010-12-01 / 2011-12-09	Giới hạn lịch sử
catchup / depends_on_past	False / False	Snapshot độc lập, tránh tạo hàng loạt

Tên	Giá trị	Nơi/quy tắc
max_active_runs / max_active_tasks	1 / 2	Concurrency cấp DAG
Spark master / deploy mode	local[2] / client	Compose connection và operator
Driver heap / shuffle partitions	3g / 8	spark_job / run_stage
Spark pool	retail_spark; 1 slot mỗi task	Pool database đã tạo 1 slot
Source Param	online-retail-v1	Token regex 1–80 ký tự
rfm_k	Mặc định 2; từ 2 đến 20	Còn cần đủ khách/vector phân biệt
churn_days	Mặc định 90; từ 1 đến 3650	Quy tắc inactivity
business_date	Manual bắt buộc; YYYY-MM-DD	C = D + 1 ngày
KMeans seed	42	pyspark_rfm.py
Row IQR	3×IQR; minimum 30	build_output defaults
Order threshold	mean + 3×sample std; minimum 30	assess_orders defaults; không phải UI Params
Rule/schema versions	retail-v2 / 1	retail_contracts.py
Trusted data root	/opt/airflow/data	RETAIL_DATA_ROOT optional deployment env
Metadata size guard	2 MiB	read_json
Notification	Local logs	Không cấu hình SMTP/Slack

Phụ lục B. Từ điển dữ liệu đầu ra thực tế

Các schema dưới được đọc từ tám dataset Parquet của published cutoff 2011-12-10. Nullable là metadata schema khi đọc Parquet, không đồng nghĩa business validator cho phép mọi cột null. Quy tắc chất lượng chi tiết nằm ở chương 10–12. Các struct/array được khai triển để giải thích trường con.

audit / flagged_orders

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
invoice_key	string	Có	InvoiceNo trim/upper dùng grouping
line_count	long	Có	Số dòng trong invoice
invalid_lines	long	Có	Số dòng làm tổng đơn không đủ điều kiện
customer_ids	long	Có	Số ID khách khác nhau không rỗng
countries	long	Có	Số Country khác nhau
invoice_days	long	Có	Số ngày giao dịch khác nhau
eligible	boolean	Có	Invoice đáp ứng điều kiện nghiệp vụ để lấy tổng
order_total	decimal(33,2)	Có	Tổng tiền làm tròn; null nếu invoice không hợp lệ
reason	string	Có	Lý do không đủ điều kiện
reference_n	integer	Có	Số invoice đủ điều kiện trong baseline
reference_mean	double	Có	Mean tổng đơn baseline
reference_stddev	double	Có	Sample standard deviation
upper_bound	double	Có	Ngưỡng trên
currency	string	Có	GBP
multiplier	double	Có	Hệ số độ lệch chuẩn
run_date	date	Có	Cutoff exclusive C
reference_mode	string	Có	retrospective_before_run_date
status	string	Có	flagged / not_flagged / not_assessable

audit / order_assessments

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
invoice_key	string	Có	InvoiceNo trim/upper dùng grouping
line_count	long	Có	Số dòng trong invoice
invalid_lines	long	Có	Số dòng làm tổng đơn không đủ điều kiện
customer_ids	long	Có	Số ID khách khác nhau không rỗng
countries	long	Có	Số Country khác nhau
invoice_days	long	Có	Số ngày giao dịch khác nhau
eligible	boolean	Có	Invoice đáp ứng điều kiện nghiệp vụ để lấy tổng
order_total	decimal(33,2)	Có	Tổng tiền làm tròn; null nếu invoice không hợp lệ
reason	string	Có	Lý do không đủ điều kiện
reference_n	integer	Có	Số invoice đủ điều kiện trong baseline
reference_mean	double	Có	Mean tổng đơn baseline
reference_stddev	double	Có	Sample standard deviation
upper_bound	double	Có	Ngưỡng trên

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
currency	string	Có	GBP
multiplier	double	Có	Hệ số độ lệch chuẩn
run_date	date	Có	Cutoff exclusive C
reference_mode	string	Có	retrospective_before_run_date
status	string	Có	flagged / not_flagged / not_assessable

audit / row_assessments

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
InvoiceNo	string	Có	Mã hóa đơn gốc
StockCode	string	Có	Mã sản phẩm/phí gốc
Description	string	Có	Mô tả gốc
Quantity	integer	Có	Số lượng đã parse
InvoiceDate	timestamp	Có	Timestamp giao dịch
UnitPrice	double	Có	Đơn giá gốc sau parse
CustomerID	string	Có	ID khách dạng string
Country	string	Có	Quốc gia nguồn
run_date	date	Có	Cutoff exclusive C
reference_mode	string	Có	retrospective_before_run_date
iqr_multiplier	double	Có	Hệ số IQR
min_samples	integer	Có	Số mẫu sản phẩm tối thiểu
line_value	double	Có	Quantity × UnitPrice hữu hạn
data_quality_flags	array<string>	Có	Danh sách vấn đề trường dữ liệu
context_flags	array<string>	Có	Các cờ bối cảnh
check_results	array<struct>	Có	Chín kết quả check có cấu trúc
anomaly_flags	array<string>	Có	Tên check flagged
assessment_status	string	Có	Trạng thái tổng của dòng

Các trường bên trong mỗi phần tử check_results:

Trường con	Kiểu	Ý nghĩa
rule	string	Tên kiểm tra
status	string	flagged / not_flagged / not_applied
reason	string	Lý do không đủ điều kiện
observed_value	double	Giá trị quan sát
reference_count	long	Số mẫu tham chiếu
q1	double	Tứ phân vị 25%
q3	double	Tứ phân vị 75%
upper_bound	double	Ngưỡng trên

etl / audit_input

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
InvoiceNo	string	Có	Mã hóa đơn gốc
StockCode	string	Có	Mã sản phẩm/phí gốc
Description	string	Có	Mô tả gốc
Quantity	integer	Có	Số lượng đã parse
InvoiceDate	timestamp	Có	Timestamp giao dịch
UnitPrice	double	Có	Đơn giá gốc sau parse
CustomerID	string	Có	ID khách dạng string
Country	string	Có	Quốc gia nguồn

etl / parse_errors

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
raw_InvoiceNo	string	Có	Giá trị chuỗi nguồn trước parse: InvoiceNo
raw_StockCode	string	Có	Giá trị chuỗi nguồn trước parse: StockCode
raw_Description	string	Có	Giá trị chuỗi nguồn trước parse: Description
raw_Quantity	string	Có	Giá trị chuỗi nguồn trước parse: Quantity
raw_InvoiceDate	string	Có	Giá trị chuỗi nguồn trước parse: InvoiceDate
raw_UnitPrice	string	Có	Giá trị chuỗi nguồn trước parse: UnitPrice
raw_CustomerID	string	Có	Giá trị chuỗi nguồn trước parse: CustomerID
raw_Country	string	Có	Giá trị chuỗi nguồn trước parse: Country
InvoiceNo	string	Có	Mã hóa đơn gốc
StockCode	string	Có	Mã sản phẩm/phí gốc
Description	string	Có	Mô tả gốc
Quantity	integer	Có	Số lượng đã parse
InvoiceDate	timestamp	Có	Timestamp giao dịch
UnitPrice	double	Có	Đơn giá gốc sau parse
CustomerID	string	Có	ID khách dạng string
Country	string	Có	Quốc gia nguồn

etl / transactions

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
InvoiceNo	string	Có	Mã hóa đơn gốc
StockCode	string	Có	Mã sản phẩm/phí gốc
Description	string	Có	Mô tả gốc
Quantity	integer	Có	Số lượng đã parse
InvoiceDate	timestamp	Có	Timestamp giao dịch
UnitPrice	double	Có	Đơn giá gốc sau parse
CustomerID	string	Có	ID khách dạng string
Country	string	Có	Quốc gia nguồn

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
year	integer	Có	Partition năm
month	integer	Có	Partition tháng

rfm / cluster_profile

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
Cluster	integer	Có	ID KMeans tùy ý
customers	long	Có	Số khách trong cụm
mean_Recency	double	Có	Trung bình Recency trong cụm
mean_Frequency	double	Có	Trung bình Frequency trong cụm
mean_Monetary	decimal(37,6)	Có	Trung bình Monetary trong cụm

rfm / customers

Cột	Kiểu Parquet/Spark	Nullable	Ý nghĩa
CustomerID	string	Có	ID khách dạng string
run_date	date	Có	Cutoff exclusive C
LastPurchaseDate	timestamp	Có	Lần mua gần nhất trước C
Recency	integer	Có	Số ngày từ last purchase đến C
Frequency	long	Có	Số invoice phân biệt
Monetary	decimal(33,2)	Có	Tổng tiền dương theo nhánh clean
R_score	integer	Có	Điểm tương đối Recency 1–5
F_score	integer	Có	Điểm Frequency 1–5
M_score	integer	Có	Điểm Monetary 1–5
RFM_score	string	Có	Chuỗi nối R/F/M score
Cluster	integer	Có	ID KMeans tùy ý
is_high_value	boolean	Có	M_score >= 4
is_inactive	boolean	Có	Recency >= churn_days
observed_history_days	integer	Có	Độ dài lịch sử nguồn đến cutoff
churn_threshold_days	integer	Có	Ngưỡng inactivity được chọn
segment_rule_version	string	Có	Phiên bản quy tắc nhãn
churn_assessment_status	string	Có	insufficient_history / inactivity_proxy / not_flagged
segment	string	Có	Nhãn nghiệp vụ ưu tiên theo quy tắc

Phụ lục C. Danh mục hàm và điểm vào của code

Danh mục này được tạo từ AST các file runtime trong repository tại thời điểm dựng tài liệu. Nó giúp tìm đúng nơi sửa/đọc; không phải một bản copy toàn bộ source. Việc liệt kê helper nội bộ không biến nó thành API công khai ổn định.

dags/ecommerce_etl_dag.py

Hàm	Dòng nguồn	Trách nhiệm
spark_job	33	Tạo operator Spark với cấu hình chung và timeout của stage.

dags/retail_support/tasks.py

Hàm	Dòng nguồn	Trách nhiệm
business_date	23	Tách quy tắc manual và scheduled/backfill; chặn ngày không nhất quán.
execute	48	Chờ manifest bằng lớp cha rồi kiểm tra nguồn và tạo context.
notify_completion	70	Xác minh stage manifests, công bố kết quả và log summary.
log_failure	79	Ghi dag/task/run/attempt khi task thất bại.
log_retry	90	Ghi sự kiện task thử lại.
log_deadline_miss	94	Đọc context deadline giới hạn và ghi cảnh báo.

scripts/retail_contracts.py

Hàm	Dòng nguồn	Trách nhiệm
data_root	27	Lấy root đáng tin cậy từ môi trường hoặc mặc định.
safe_path	32	Resolve và chặn đường dẫn thoát root.
token	40	Kiểm tra nhãn source version bằng regex.
iso_date	48	Yêu cầu ngày ISO chuẩn.
days	57	Sinh các ngày liên tiếp bao gồm hai biên.
digest	65	Băm file SHA-256 theo chunk.
json_hash	73	Fingerprint JSON sắp khóa.
read_json	79	Đọc metadata có guard kích thước.
atomic_json	86	Ghi tạm/fsync rồi thay file metadata.
code_version	99	Fingerprint năm file business/shared code.
validate_landing	112	Kiểm tra version, coverage, paths, bytes, header và tùy chọn full hash.
context_file	150	Định vị context từ run_key hex hợp lệ.
build_context	156	Cố định source/code/date/params của DagRun; từ chối thay đổi.
load_context	187	Đọc context và xác minh fingerprint code hiện tại.
stage_file	198	Định vị manifest của ETL/RFM/audit.
load_stage	204	Kiểm tra context hash, complete và marker output.
publication_lock	217	Lock exclusive theo cutoff, giải phóng ở finally.

Hàm	Dòng nguồn	Trách nhiệm
publish_completion	229	Chỉ công bố ba stage nhất quán với cùng ETL generation.

scripts/prepare_daily_landing.py

Hàm	Dòng nguồn	Trách nhiệm
prepare	23	Bootstrap CP1252 → daily UTF-8, round-trip và manifest bất biến.

scripts/retail_pipeline.py

Hàm	Dòng nguồn	Trách nhiệm
finite	28	Biểu thức nhận số khác null, NaN và infinity.
read_landing	36	Đọc CSV rõ schema và giữ raw values của lỗi parse.
customer_labels	73	Gán high-value, inactivity và segment theo thứ tự ưu tiên.
assess_orders	110	Tổng hóa đơn có eligibility/reason và ngưỡng mean+3s.
counts	204	Collect bảng đếm nhóm nhỏ để ghi metrics.
parquet	211	Ghi dataset mới và yêu cầu marker _SUCCESS.
run_clean	220	Thực thi ETL pipeline mode, kiểm tra nguồn và ghi ba output.
run_rfm	283	Thực thi RFM pipeline mode từ ETL manifest.
run_audit	338	Thực thi row/order audit và đối soát số lượng.
run_stage	380	Tạo generation, SparkSession, dispatch và commit manifest.
pipeline_main	428	CLI --pipeline-context cho từng legacy entry point.

scripts/pyspark_clean.py

Hàm	Dòng nguồn	Trách nhiệm
read_transactions	48	Đọc CSV legacy bằng schema giao dịch tường minh.
prepare_datasets	59	Dedup và tách clean-RFM / audit với hai null policies.
write_datasets	78	Ghi hai thư mục legacy bằng overwrite.
run_cleaning_job	89	Ghép đọc, transform và ghi cleaning legacy.
parse_args	102	Kiểm tra tham số CLI của script.
main	120	Điểm vào CLI và vòng đời tài nguyên/error handling.

scripts/pyspark_rfm.py

Hàm	Dòng nguồn	Trách nhiệm
parse_args	31	Kiểm tra tham số CLI của script.
validate_curated_contract	84	Kiểm tra schema và giá trị nhánh RFM trước tổng hợp.

Hàm	Dòng nguồn	Trách nhiệm
select_history	158	Lọc lịch sử theo cutoff; chính sách null-date tùy script.
compute_rfm	184	Aggregation theo khách: last purchase, distinct orders, money.
add_rfm_scores	234	Window percent_rank giữ ties và tạo điểm 1–5.
prepare_kmeans_features	272	log1p ba chỉ số R/F/M.
kmeans_feature_pipeline	281	VectorAssembler + StandardScaler thống nhất.
add_rfm_segment	297	Fit KMeans có kiểm tra khách/vector/cụm occupied.
validate_rfm_output	354	Kiểm tra metrics, scores, cutoff và cluster range.
write_rfm	416	Ghi RFM legacy theo ngày và kiểm tra đường dẫn.
main	431	Điểm vào CLI và vòng đời tài nguyên/error handling.

scripts/pyspark_anomalies.py

Hàm	Dòng nguồn	Trách nhiệm
parse_args	38	Kiểm tra tham số CLI của script.
validate_paths	58	Chặn input/output overlap trước ghi legacy.
validate_input_contract	69	Kiểm tra tám cột và đúng kiểu giao dịch cho audit.
select_history	81	Lọc lịch sử theo cutoff; chính sách null-date tùy script.
_finite	88	Biểu thức kiểm tra số hữu hạn dùng trong detector.
_normalized	92	Chuẩn hóa chuỗi nội bộ, không thay cột gốc.
_flags	99	Tạo array tên flags có điều kiện đúng.
_check	106	Tạo struct kết quả một check với reason và evidence.
add_base_columns	126	Cột nội bộ, line_value và DQ flags.
add_description_context	165	Tên tham chiếu cùng stock và keyword context.
add_invoice_checks	198	Sáu quy tắc nghiệp vụ từ nhóm invoice.
add_product_iqr_checks	246	Ba rule IQR với eligibility/số mẫu/reason.
build_output	291	Ghép chín check, flags và trạng thái tổng của dòng.
validate_output	323	Bảo toàn multiset, thứ tự rule và status/cutoff.
write_results	341	Ghi partition anomaly legacy của một ngày.
_log_results	348	Log bảng tổng hợp trạng thái/rule/reason nhỏ.
main	367	Điểm vào CLI và vòng đời tài nguyên/error handling.

scripts/select_kmeans_k.py

Hàm	Dòng nguồn	Trách nhiệm
parse_args	35	Kiểm tra tham số CLI của script.
score_k_range	52	Fit/evaluate các k trên cùng không gian feature.
write_report	70	Xuất k_scores.csv và biểu đồ nếu có matplotlib.
main	106	Điểm vào CLI và vòng đời tài nguyên/error handling.

Phụ lục D. Cấu hình triển khai và DAG đầy đủ

D.1 Dockerfile

```
FROM apache/airflow:3.3.1

USER root
RUN apt-get update \
    && apt-get install -y --no-install-recommends openjdk-17-jre-headless \
    && apt-get autoremove -yqq --purge \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

USER airflow
ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
COPY requirements.txt /
RUN pip install --no-cache-dir "apache-airflow==${AIRFLOW_VERSION}" -r /requirements.txt
```

D.2 Docker Compose

Bản chép dưới lấy từ source Compose. Giá trị default cho API JWT đã thay bằng một placeholder trong tài liệu; người chạy phải tự tạo secret trong .env. Tài khoản database local trong cấu hình không phải khuyến nghị bảo mật production. Không có secret thực tế từ .env được chèn vào bản Word.

```
name: ecommerce-airflow

x-airflow-common: &airflow-common
  build: .
  image: ecommerce-airflow:3.3.1
  environment: &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__CORE__AUTH_MANAGER: airflow.providers.fab.auth_manager.fab_auth_manager.FabAuthManager
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CORE__FERNET_KEY: ${AIRFLOW_FERNET_KEY:-}
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: "true"
    AIRFLOW__CORE__LOAD_EXAMPLES: "false"
    AIRFLOW__CORE__EXECUTION_API_SERVER_URL: http://airflow-apiserver:8080/execution/
    AIRFLOW__API__AUTH_JWT_SECRET: ${AIRFLOW_API_JWT_SECRET:-TU_TAO_JWT_SECRET}
    AIRFLOW__CONN__SPARK_DEFAULT: '{"conn_type":"spark","host":"local[2]"}'
    AIRFLOW__CONN__RETAIL_LANDING: '{"conn_type":"fs","extra":{"path":"/opt/airflow/data/raw/landing"}}'
    PYTHONPATH: /opt/airflow:/opt/airflow/scripts:/opt/airflow/dags
  volumes:
    - ./dags:/opt/airflow/dags
    - ./scripts:/opt/airflow/scripts
    - ./tests:/opt/airflow/tests:ro
    - ./data:/opt/airflow/data
    - ./logs:/opt/airflow/logs
    - ./pyproject.toml:/opt/airflow/pyproject.toml:ro
  user: "${AIRFLOW_UID:-50000}:0"
  depends_on:
    postgres:
      condition: service_healthy

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-db-volume:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      timeout: 5s
      retries: 5
      restart: unless-stopped

  airflow-init:
    <<: *airflow-common
```

```

command: version
environment:
  <<: *airflow-common-env
  _AIRFLOW_DB_MIGRATE: "true"
  _AIRFLOW_WWW_USER_CREATE: "true"
  _AIRFLOW_WWW_USER_USERNAME: ${AIRFLOW_ADMIN_USERNAME:-airflow}
  _AIRFLOW_WWW_USER_PASSWORD: ${AIRFLOW_ADMIN_PASSWORD:-airflow}
restart: "no"

airflow-apiserver:
  <<: *airflow-common
  command: api-server
  ports:
    - "8080:8080"
  healthcheck:
    test: ["CMD", "curl", "--fail", "http://localhost:8080/api/v2/monitor/health"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  depends_on:
    postgres:
      condition: service_healthy
    airflow-init:
      condition: service_completed_successfully
  restart: unless-stopped

airflow-scheduler:
  <<: *airflow-common
  command: scheduler
  healthcheck:
    test: ["CMD-SHELL", "airflow jobs check --job-type SchedulerJob --hostname \"${HOSTNAME}\""]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  depends_on:
    postgres:
      condition: service_healthy
    airflow-init:
      condition: service_completed_successfully
  restart: unless-stopped

airflow-dag-processor:
  <<: *airflow-common
  command: dag-processor
  healthcheck:
    test: ["CMD-SHELL", "airflow jobs check --job-type DagProcessorJob --hostname \"${HOSTNAME}\""]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  depends_on:
    postgres:
      condition: service_healthy
    airflow-init:
      condition: service_completed_successfully
  restart: unless-stopped

volumes:
  postgres-db-volume:

```

D.3 DAG chính thức

```

"""Daily historical retail pipeline: six tasks with versioned Spark outputs."""

from datetime import timedelta

import pendulum
from airflow.providers.apache.spark.operators.spark_submit import (
    SparkSubmitOperator,
)
from airflow.providers.standard.operators.empty import EmptyOperator
from airflow.providers.standard.operators.python import PythonOperator
from airflow.sdk import (

```

```

    DAG,
    CronDataIntervalTimetable,
    DeadlineAlert,
    DeadlineReference,
    Param,
    SyncCallback,
)

from retail_support.tasks import (
    LandingValidationSensor,
    log_deadline_miss,
    log_failure,
    log_retry,
    notify_completion,
)

CONTEXT_PATH = (
    "{{ ti.xcom_pull(task_ids='validate_raw_data')['context_path'] }}"
)

def spark_job(task_id, script, minutes):
    """The pool bounds concurrent JVMs; the Connection sets local[2]."""
    return SparkSubmitOperator(
        task_id=task_id,
        application=f"/opt/airflow/scripts/{script}",
        application_args=["--pipeline-context", CONTEXT_PATH],
        conn_id="spark_default",
        deploy_mode="client",
        driver_memory="3g",
        conf={"spark.sql.session.timeZone": "UTC"},
        pool="retail_spark",
        pool_slots=1,
        durable=False,
        do_xcom_push=False,
        execution_timeout=timedelta(minutes=minutes),
    )

with DAG(
    dag_id="ecommerce_etl_dag",
    description="Retail ETL, customer RFM and retrospective order audit",
    schedule=CronDataIntervalTimetable("0 0 * * *", timezone="UTC"),
    start_date=pendulum.datetime(2010, 12, 1, tz="UTC"),
    end_date=pendulum.datetime(2011, 12, 9, tz="UTC"),
    catchup=False,
    is_paused_upon_creation=True,
    max_active_runs=1,
    max_active_tasks=2,
    dagrun_timeout=timedelta(minutes=90),
    deadline=DeadlineAlert(
        reference=DeadlineReference.DAGRUN_QUEUED_AT,
        interval=timedelta(minutes=60),
        callback=SyncCallback(log_deadline_miss),
    ),
    params={
        "business_date": Param(
            None,
            type=["null", "string"],
            format="date",
            description="Manual source day, e.g. 2011-12-09 (cutoff +1 day)",
        ),
        "source_version": Param(
            "online-retail-v1",
            type="string",
            pattern=r"^[A-Za-z0-9_-]{1,80}$",
        ),
        "rfm_k": Param(2, type="integer", minimum=2, maximum=20),
        "churn_days": Param(90, type="integer", minimum=1, maximum=3650),
    },
    default_args={
        "owner": "retail-data-team",
        "depends_on_past": False,
        "retries": 1,
        "retry_delay": timedelta(minutes=5),
        "retry_exponential_backoff": True,
        "max_retry_delay": timedelta(minutes=15),
        "on_failure_callback": log_failure,
    },
)

```

```

        "on_retry_callback": log_retry,
    },
    tags=["retail", "pyspark", "daily", "historical-demo"],
    doc_md=""
)
"""
### Retail pipeline
Prepare landing and the retail_spark pool first (docs/SETUP.md).
Manual trigger: business_date=2011-12-09, rfm_k=2, churn_days=90.
The exclusive analysis cutoff is 2011-12-10. The daily timetable is bounded
to historical data: use explicit manual runs/backfills, not today's date.
Success requires both analytics branches and the publication summary.
Outputs are versioned; original CSV and legacy Parquet are preserved.
"""
) as dag:
    start_pipeline = EmptyOperator(
        task_id="start_pipeline",
        retries=0,
        execution_timeout=timedelta(minutes=1),
    )
    validate_raw_data = LandingValidationSensor(
        task_id="validate_raw_data",
        filepath="manifest.json",
        fs_conn_id="retail_landing",
        mode="reschedule",
        poke_interval=60,
        timeout=1800,
        soft_fail=False,
        deferrable=False,
        retries=0,
        execution_timeout=timedelta(minutes=2),
        do_xcom_push=True,
    )
    submit_pyspark_etl = spark_job(
        "submit_pyspark_etl", "pyspark_clean.py", 20
    )
    compute_rfm_metrics = spark_job(
        "compute_rfm_metrics", "pyspark_rfm.py", 20
    )
    detect_anomalies = spark_job(
        "detect_anomalies", "pyspark_anomalies.py", 30
    )
    completion = PythonOperator(
        task_id="notify_completion",
        python_callable=notify_completion,
        trigger_rule="all_success",
        execution_timeout=timedelta(minutes=2),
        do_xcom_push=False,
    )

    start_pipeline >> validate_raw_data >> submit_pyspark_etl
    submit_pyspark_etl >> [compute_rfm_metrics, detect_anomalies]
    [compute_rfm_metrics, detect_anomalies] >> completion

```

D.4 Cấu hình lint

```

[tool.ruff]
target-version = "py312"
line-length = 79

[tool.ruff.lint]
select = ["E4", "E7", "E9", "F", "W", "E501"]

# Existing research code retains formatting; new pipeline code is checked.
[tool.ruff.lint.per-file-ignores]
"scripts/pyspark_*.py" = ["E501", "E402"]
"tests/test_pyspark_*.py" = ["E501", "E402"]
"scripts/select_kmeans_k.py" = ["E501"]

```

Phụ lục E. Quy trình cập nhật tài liệu

Nội dung biên soạn nằm trong [docs/handbook/SO_TAY_KIEN_TRUC_VI.md](#); script [build_handbook.py](#) tạo hình, bảng tự động và DOCX; [render_word.ps1](#) dùng Microsoft Word để cập nhật mục lục, phân trang và xuất PDF. Dependency dựng tài liệu tách trong requirements-docs.txt, không thêm vào image pipeline. Người chỉ chạy ETL không cần cài các dependency tạo Word.

Khi code hoặc kết quả thay đổi, cập nhật chương liên quan, lấy bằng chứng run mới nếu có, rồi dựng lại cả Word/PDF. Không thay các số liệu cũ bằng giả định. Đọc lại trang mục lục, hình, bảng dài, code và các trang cuối; đối chiếu checksum/version và ma trận yêu cầu. Bản PDF giúp xem nhanh bố cục, còn DOCX là bản chỉnh sửa chính.